**BỘ GIÁO DỤC VÀ ĐÀO TẠO**

**TRƯỜNG ĐẠI HỌC NHA TRANG**

**KHOA CÔNG NGHỆ THÔNG TIN**

_____* *_____

**ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC**

# Xây dựng nền tảng quản lý nhà trọ thông minh (Nhatroso Platform)

**Giảng viên hướng dẫn** : TS. Nguyễn Đình Hưng

**Sinh viên thực hiện** : Đinh Nhật Bảo

**MSSV** : 64130107

**Khánh Hòa, 04/2026**

# Lời cam đoan

Em tên là Đinh Nhật Bảo, em xin cam đoan rằng đồ án tốt nghiệp với đề tài "Xây dựng nền tảng quản lý nhà trọ thông minh (Nhatroso Platform)" là công trình nghiên cứu và thực hiện độc lập của cá nhân em dưới sự hướng dẫn chuyên môn của TS. Nguyễn Đình Hưng.

Toàn bộ quá trình thực hiện đồ án từ các giai đoạn phân tích chiến lược, đặc tả yêu cầu, thiết kế kiến trúc hệ thống cho đến các giai đoạn phát triển, kiểm thử và bàn giao đều tuân thủ nghiêm ngặt lộ trình phát triển sản phẩm chuyên nghiệp kéo dài 16 tuần. Trong quá trình thực hiện, em có sử dụng một số thư viện và công cụ mã nguồn mở phổ biến nhằm hỗ trợ phát triển hệ thống. Các thành phần này đều được trích dẫn và ghi nhận đầy đủ trong phần tài liệu tham khảo, đồng thời đều là các phần mềm miễn phí hoặc được sử dụng theo giấy phép hợp lệ, không vi phạm bản quyền.

Em xin chịu hoàn toàn trách nhiệm về tính xác thực của các nội dung trích dẫn và tài liệu tham khảo được sử dụng trong báo cáo này. Mọi sự hỗ trợ từ các cá nhân hay tổ chức khác trong quá trình thực hiện đồ án đã được ghi nhận và cảm ơn đầy đủ. Nếu phát hiện bất kỳ sự gian lận nào so với những lời cam đoan trên, em xin chịu hoàn toàn trách nhiệm trước Hội đồng chấm Đồ án và Nhà trường.

**Người cam đoan**

**Đinh Nhật Bảo**

# Lời cảm ơn

Lời đầu tiên, em xin bày tỏ lòng biết ơn chân thành và sâu sắc nhất tới **TS. Nguyễn Đình Hưng** – giảng viên hướng dẫn đã tận tâm định hướng, truyền đạt những kinh nghiệm quý báu và luôn theo sát, hỗ trợ em vượt qua những khó khăn trong suốt lộ trình thực hiện đồ án. Những lời góp ý chuyên môn của thầy không chỉ giúp hoàn thiện hệ thống Nhatroso Platform mà còn giúp em rèn luyện tư duy lập trình và quản lý dự án chuyên nghiệp.

Em xin trân trọng cảm ơn Ban Giám hiệu, cùng toàn thể quý thầy cô khoa **Công nghệ Thông tin** – Trường Đại học Nha Trang, những người đã truyền dạy tri thức và tạo mọi điều kiện thuận lợi nhất cho em trong suốt quá trình học tập tại trường. Những kiến thức nền tảng và kỹ năng được trau dồi tại nhà trường chính là bệ phóng quan trọng nhất để em có thể tự tin triển khai một hệ thống thực tế.

Cuối cùng, em xin gửi lời cảm ơn sâu sắc nhất tới gia đình, những người luôn là chỗ dựa vững chắc về mặt tinh thần, và các bạn đồng môn đã luôn sát cánh, chia sẻ và khích lệ em trong suốt quá trình hoàn thành Đồ án tốt nghiệp này.

# Tóm tắt nội dung

Quản lý nhà trọ tại Việt Nam hiện vẫn đối mặt với nhiều thách thức lớn do phụ thuộc vào các phương pháp thủ công như ghi chép sổ sách và bảng tính Excel. Những phương thức này không chỉ gây tốn kém thời gian mà còn dẫn đến nhiều sai sót trong việc ghi nhận chỉ số điện nước, gây thiếu minh bạch và phát sinh tranh chấp giữa chủ nhà và người thuê, đồng thời làm chậm dòng tiền vận hành.

Đồ án này nghiên cứu và phát triển **Nhatroso Platform** – một giải pháp số hóa toàn diện nhằm hiện đại hóa quy trình quản lý nhà trọ. Nền tảng cung cấp bộ công cụ tối ưu bao gồm: quy trình ghi nhận chỉ số điện nước minh bạch bằng hình ảnh trực tiếp trên thiết bị di động kết hợp cùng công nghệ **OCR** để tự động hóa việc nhận diện chỉ số, hệ thống tự động hóa lập hóa đơn hàng tháng và cơ chế đối soát thanh toán trực tuyến qua QR code với thông báo Webhook thời gian thực.

Về mặt kỹ thuật, hệ thống được thiết kế theo kiến trúc *Modular Monolith* nhằm đảm bảo tính linh hoạt và khả năng mở rộng. Core API được xây dựng bằng ngôn ngữ lập trình **Rust** với framework **Loco.rs**, mang lại hiệu suất vượt trội và tính an toàn bộ nhớ cao. Dữ liệu được quản trị thông qua **SeaORM** trên nền tảng **PostgreSQL**. Giao diện quản lý tập trung (Web Dashboard) được phát triển bằng **Next.js 16**, kết hợp cùng ứng dụng di động cho người thuê (Mobile App) xây dựng trên nền tảng **Expo**, đảm bảo trải nghiệm người dùng mượt mà và đồng nhất.

Kết quả thực nghiệm cho thấy **Nhatroso Platform** giúp giảm thiểu đáng kể thời gian xử lý nghiệp vụ hàng tháng, loại bỏ các sai lệch trong ghi nhận chỉ số và cung cấp cho chủ nhà cái nhìn tổng quan về doanh thu cũng như tình trạng bất động sản thông qua hệ thống báo cáo Dashboard trực quan. Hệ thống không chỉ giải quyết các vấn đề vận hành hiện tại bằng sức mạnh của trí tuệ nhân tạo (OCR) mà còn đặt nền móng cho việc ứng dụng công nghệ **IoT** vào giám sát chỉ số thông minh trong tương lai.

# Mục lục

**Lời cam đoan** . . . . . **i**

**Lời cảm ơn** . . . . . **ii**

**Tóm tắt nội dung** . . . . . **iii**

**Danh mục từ viết tắt** . . . . . **ix**

**1 Giới thiệu** . . . . . **1**
1.1 Đặt vấn đề . . . . . 1
1.2 Mục tiêu đề tài . . . . . 1
1.3 Đối tượng và phạm vi nghiên cứu . . . . . 2
1.4 Quy trình phát triển và công cụ sử dụng . . . . . 3
1.5 Ý nghĩa đồ án . . . . . 4
1.6 Bố cục đồ án . . . . . 5

**2 Cơ sở lý thuyết** . . . . . **6**
2.1 Khái niệm liên quan . . . . . 6
2.1.1 Hệ thống quản lý (Management System) . . . . . 6
2.1.2 Kiến trúc đa khách thuê (Multi-tenancy) . . . . . 6
2.1.3 Số hóa quy trình quản lý nhà trọ . . . . . 8
2.2 Công nghệ sử dụng . . . . . 9
2.2.1 Ngôn ngữ lập trình Rust . . . . . 9
2.2.2 Framework Loco.rs . . . . . 9
2.2.3 Next.js 16 . . . . . 10
2.2.4 Expo và React Native . . . . . 10
2.2.5 Cơ sở dữ liệu PostgreSQL và SeaORM . . . . . 11
2.3 Mô hình và Thuật toán . . . . . 11
2.3.1 Kỹ thuật nhận dạng ký tự quang học (OCR) với AI . . . . . 11
2.3.2 Internet of Things (IoT) và Giao thức MQTT . . . . . 12
2.4 Quy trình phát triển phần mềm . . . . . 12
2.4.1 Vòng đời phát triển phần mềm (SDLC) . . . . . 12
2.4.2 Mô hình Agile Scrum . . . . . 14
2.5 Kiến trúc hệ thống và Bảo mật . . . . . 16
2.5.1 Kiến trúc Modular Monolith . . . . . 16

2.5.2 Kiến trúc Phân quyền RBAC . . . . . 17
2.6 Triển khai và Tích hợp liên tục (CI/CD) . . . . . 18
2.6.1 Tích hợp liên tục (Continuous Integration - CI) . . . . . 19
2.6.2 Triển khai liên tục (Continuous Delivery/Deployment - CD) . . . . 19

**3 Phân tích yêu cầu hệ thống . . . . . 21**
3.1 Phân tích stakeholder . . . . . 21
3.1.1 Chi tiết các nhóm stakeholder . . . . . 21
3.2 Yêu cầu chức năng . . . . . 22
3.2.1 Quản lý người dùng và phân quyền (Authentication & RBAC) . . . 22
3.2.2 Quản lý hạ tầng bất động sản (Property Management) . . . . . 22
3.2.3 Quản lý dịch vụ và bảng giá . . . . . 23
3.2.4 Quản lý hợp đồng và người thuê (Contract Management) . . . . . 23
3.2.5 Quản lý chỉ số điện nước (Meter Reading Management) . . . . . 23
3.2.6 Quản lý hóa đơn và thanh toán (Invoice & Payment Management) . 24
3.3 Yêu cầu phi chức năng . . . . . 24
3.3.1 Tính bảo mật (Security) . . . . . 24
3.3.2 Hiệu năng và Khả năng mở rộng . . . . . 25
3.3.3 Độ tin cậy và Tính nhất quán (Reliability & Integrity) . . . . . 25
3.3.4 Khả năng bảo trì và Tiến hóa (Maintainability) . . . . . 25
3.3.5 Trải nghiệm người dùng và Tính tương thích (Usability) . . . . . 26

**4 Thiết kế hệ thống . . . . . 27**
4.1 Kiến trúc tổng thể . . . . . 27
4.1.1 Sơ đồ Use Case (Các ca sử dụng cốt lõi) . . . . . 28
4.2 Thiết kế cơ sở dữ liệu . . . . . 29
4.2.1 Sơ đồ quan hệ thực thể . . . . . 29
4.3 Thiết kế API . . . . . 30
4.4 Thiết kế luồng nghiệp vụ quan trọng . . . . . 30
4.4.1 Luồng nhận diện tự động và phê duyệt chỉ số . . . . . 30
4.4.2 Quy trình tính hóa đơn (Billing Logic) . . . . . 32
4.5 Thiết kế luồng Tích hợp và Triển khai tự động (CI/CD) . . . . . 33

**5 Cài đặt và triển khai . . . . . 35**
5.1 Công nghệ sử dụng . . . . . 35
5.1.1 Backend API (Rust - Loco.rs) . . . . . 35
5.1.2 Owner Dashboard (Next.js) . . . . . 35
5.1.3 Tenant Mobile App (Expo) . . . . . 35
5.2 Hạ tầng và Triển khai Đám mây . . . . . 36
5.3 Cấu trúc module Backend (Loco.rs) . . . . . 36

5.4 Bảo mật và Phân quyền . . . . . 36

**6 Kiểm thử hệ thống . . . . . 38**

6.1 Chiến lược kiểm thử . . . . . 38

6.2 Chiến lược kiểm thử . . . . . 38

6.3 Unit Test và Integration Test trong Rust . . . . . 38

6.4 Kiểm thử luồng nghiệp vụ đầu cuối (E2E) . . . . . 39

**7 Kết luận . . . . . 40**

7.1 Kết quả đạt được . . . . . 40

7.2 Hạn chế và tồn tại . . . . . 40

7.3 Hướng phát triển . . . . . 41

# Danh sách hình vẽ


2.1 Mô hình kiến trúc tổng quát của hệ thống Đa khách thuê . . . . . 7
2.2 Các giai đoạn trong vòng đời phát triển phần mềm (SDLC) . . . . . 14
2.3 Cấu trúc quy trình thực thi theo mô hình Agile Scrum . . . . . 16
2.4 Mô hình minh họa kiến trúc Modular Monolith . . . . . 17
2.5 Mô hình vòng lặp minh họa luồng CI/CD . . . . . 20

4.1 Sơ đồ kiến trúc tổng thể của hệ thống Nhatroso Platform . . . . . 27
4.2 Sơ đồ Use Case tổng quát . . . . . 28
4.3 Sơ đồ quan hệ thực thể (ERD) . . . . . 30
4.4 Sơ đồ tuần tự luồng thông báo chỉ số bằng OCR . . . . . 32
4.5 Sơ đồ tuần tự phát hành hóa đơn tự động . . . . . 33

# Danh sách bảng


3.1 Ma trận phân tích stakeholder (Interest Matrix) . . . . . 21
3.2 Tóm tắt các chỉ số yêu cầu phi chức năng . . . . . 26

4.1 Các thành phần hệ thống (Implemented) . . . . . 29
4.2 Các bảng dữ liệu chính (Sơ đồ thực tế) . . . . . 31
4.3 Các API endpoint chính . . . . . 31

5.1 Hạ tầng triển khai (Implemented) . . . . . 36

6.1 Kịch bản integration test (Implemented) . . . . . 39

# Danh mục từ viết tắt

| Ký hiệu/Viết tắt | Ý nghĩa/Giải thích |
|---|---|
| **ACID** | Atomicity, Consistency, Isolation, Durability (Tính chất giao dịch CSDL) |
| **API** | Application Programming Interface (Giao diện lập trình ứng dụng) |
| **AWS** | Amazon Web Services (Các dịch vụ web của Amazon) |
| **BRD** | Business Requirements Document (Tài liệu yêu cầu kinh doanh) |
| **CI/CD** | Continuous Integration / Continuous Deployment (Tích hợp và triển khai liên tục) |
| **CSS** | Cascading Style Sheets (Ngôn ngữ định dạng trang web) |
| **Docker** | Nền tảng ảo hóa dạng container giúp đóng gói ứng dụng |
| **EC2** | Elastic Compute Cloud (Dịch vụ máy chủ ảo trên AWS) |
| **FCP** | First Contentful Paint (Chỉ số thời gian hiển thị nội dung đầu tiên) |
| **IoT** | Internet of Things (Internet vạn vật) |
| **JWT** | JSON Web Token (Tiêu chuẩn xác thực người dùng qua chuỗi mã hóa) |
| **MQTT** | Message Queuing Telemetry Transport (Giao thức truyền tin cho IoT) |
| **MVC** | Model-View-Controller (Mô hình kiến trúc phần mềm) |
| **OCR** | Optical Character Recognition (Nhận dạng ký tự quang học) |
| **ORM** | Object-Relational Mapping (Ánh xạ quan hệ đối tượng) |
| **PostgreSQL** | Hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở |
| **QR Code** | Quick Response Code (Mã phản hồi nhanh) |
| **RBAC** | Role-Based Access Control (Kiểm soát truy cập dựa trên vai trò) |
| **REST** | Representational State Transfer (Kiến trúc truyền tải trạng thái đại diện) |
| **RSC** | React Server Components (Thành phần React chạy phía máy chủ) |

| Ký hiệu/Viết tắt | Ý nghĩa/Giải thích |
|---|---|
| **Rust** | Ngôn ngữ lập trình hệ thống tập trung vào an toàn và hiệu năng |
| **S3** | Simple Storage Service (Dịch vụ lưu trữ đối tượng trên AWS) |
| **SDD** | Software Design Document (Tài liệu thiết kế phần mềm) |
| **SRS** | Software Requirements Specification (Tài liệu đặc tả yêu cầu phần mềm) |
| **SSR** | Server-Side Rendering (Kết xuất phía máy chủ) |
| **UI** | User Interface (Giao diện người dùng) |
| **UX** | User Experience (Trải nghiệm người dùng) |

# Chương 1

# Giới thiệu

## 1.1 Đặt vấn đề

Tại Việt Nam, thị trường nhà trọ phục vụ hàng triệu sinh viên, công nhân và người lao động. Phần lớn chủ cho thuê nhà trọ quy mô nhỏ và vừa vẫn đang quản lý bằng phương pháp truyền thống: ghi chỉ số điện nước thủ công vào sổ sách, tính tiền bằng Excel hoặc máy tính cầm tay, và thông báo hóa đơn qua nhắn tin. Quy trình này tiềm ẩn nhiều rủi ro: sai sót trong ghi nhận chỉ số, thiếu minh bạch trong tính toán dẫn đến tranh chấp với người thuê, và dữ liệu phân tán khó tổng hợp báo cáo.

Bên cạnh đó, việc theo dõi hợp đồng thuê, quản lý trạng thái phòng, và đối soát thanh toán hàng tháng đòi hỏi nhiều thời gian và công sức của chủ nhà. Người thuê phòng cũng thiếu công cụ để theo dõi hóa đơn và lịch sử thanh toán của mình một cách minh bạch và chủ động.

Sự phát triển của công nghệ nhận dạng ký tự quang học (OCR), ứng dụng web đám mây và hệ thống thanh toán số như QR code mở ra cơ hội số hóa toàn bộ quy trình quản lý nhà trọ, giúp giảm tải thao tác thủ công và gia tăng tính minh bạch. Trong giai đoạn đầu, việc số hóa quy trình thông qua bằng chứng ảnh chụp và hệ thống phê duyệt trực tuyến là bước đệm quan trọng trước khi tiến tới tự động hóa hoàn toàn bằng AI.

## 1.2 Mục tiêu đề tài

Mục tiêu tổng quát của đề tài là xây dựng một nền tảng quản lý nhà trọ toàn diện (Nhatroso Platform), giúp chuyển đổi số quy trình vận hành truyền thống sang mô hình hiện đại, tự động hóa và minh bạch. Cụ thể, đồ án tập trung hoàn thành các mục tiêu chi tiết sau:

- **Số hóa và minh bạch hóa quy trình ghi nhận dịch vụ:** Xây dựng cơ chế ghi nhận chỉ số điện, nước thông qua hình ảnh thực tế từ ứng dụng di động. Điều này không chỉ giúp giảm thiểu sai sót do nhập liệu thủ công mà còn tạo ra bằng chứng khách quan, giúp giải quyết triệt để các tranh chấp tiềm ẩn giữa chủ nhà và người thuê về chi phí sử dụng.
- **Xây dựng kiến trúc quản lý vận hành tập trung:** Thiết kế và triển khai hệ thống theo

mô hình kiến trúc Modular Monolith, cho phép quản lý tập trung toàn bộ dữ liệu về bất động sản (tòa nhà, tầng, phòng), hồ sơ cư dân và hợp đồng thuê trên một nền tảng duy nhất. Kiến trúc này đảm bảo tính nhất quán, an toàn dữ liệu và khả năng mở rộng linh hoạt trong tương lai.

- **Tự động hóa luồng tài chính và thanh toán:** Phát triển công cụ tự động tính toán hóa đơn hàng tháng dựa trên biểu phí dịch vụ động và chỉ số sử dụng thực tế. Tích hợp giải pháp thanh toán qua mã VietQR động, hỗ trợ đối soát giao dịch tự động thông qua Webhook, giúp tối ưu hóa dòng tiền và giảm tải công việc kế toán cho chủ nhà.
- **Tối ưu hóa trải nghiệm người dùng đa nền tảng:** Cung cấp hệ sinh thái ứng dụng đa nền tảng gồm Web Dashboard dành cho chủ nhà để quản lý vận hành chuyên sâu và Ứng dụng Mobile dành cho người thuê để chủ động theo dõi hóa đơn, nộp chỉ số và gửi yêu cầu hỗ trợ. Hệ thống đảm bảo tính thân thiện, dễ sử dụng và phản ứng nhanh trên mọi thiết bị.
- **Ứng dụng công nghệ hiện đại và độ tin cậy cao:** Nghiên cứu và áp dụng ngôn ngữ lập trình Rust cùng framework Loco.rs để xây dựng một hệ thống Backend có hiệu suất cao, an toàn bộ nhớ và tin cậy. Đồng thời, thiết lập quy trình triển khai hiện đại sử dụng Docker và hạ tầng điện toán đám mây AWS để đảm bảo tính sẵn sàng của dịch vụ.

## 1.3 Đối tượng và phạm vi nghiên cứu

**Đối tượng nghiên cứu:** Đề tài tập trung nghiên cứu giải pháp xây dựng hệ thống quản lý vận hành nhà trọ thông minh, dựa trên việc số hóa toàn diện các luồng nghiệp vụ giữa chủ nhà và người thuê. Trọng tâm nghiên cứu bao gồm:

- Mô hình kiến trúc Modular Monolith và khả năng cách ly dữ liệu trong môi trường đa khách thuê (Multi-tenancy).
- Quy trình tự động hóa luồng tài chính từ khâu ghi nhận chỉ số đến khâu đối soát thanh toán.
- Ứng dụng công nghệ xử lý hình ảnh và OCR để giảm thiểu sai sót trong quản lý dịch vụ.

**Phạm vi nghiên cứu:**

- **Phạm vi nghiệp vụ:** Toàn bộ vòng đời quản lý lưu trú, từ quản lý cấu trúc hạ tầng (tòa nhà, tầng, phòng), quản lý hồ sơ cư dân và hợp đồng, đến các quy trình phức tạp hơn như cấu hình biểu phí dịch vụ theo thời gian, tự động hóa tính toán công nợ và tích hợp thanh toán không tiền mặt dựa trên phản hồi Webhook từ ngân hàng.
- **Phạm vi công nghệ:** Phát triển hệ thống đa nền tảng sử dụng Web Dashboard (Next.js) cho chủ nhà và Mobile App (React Native/Expo) cho người thuê. Hệ thống Backend được xây dựng bằng Rust (Loco.rs) phục vụ các giao thức RESTful API, triển khai trên nền

tảng Docker và hạ tầng AWS.

- **Giới hạn đề tài:** Do giới hạn về thời gian và nguồn lực, đề tài không bao gồm việc nghiên cứu và tích hợp các thiết bị phần cứng IoT để đọc chỉ số tự động. Hệ thống hiện tại dừng lại ở việc số hóa quy trình thông qua bằng chứng ảnh chụp và sự can thiệp của con người trong các bước phê duyệt cuối cùng để đảm bảo độ chính xác tuyệt đối.

## 1.4 Quy trình phát triển và công cụ sử dụng

**Quy trình phát triển phần mềm:** Đề tài được thực hiện theo mô hình phát triển phần mềm **Agile/Scrum** [16], cho phép tính linh hoạt cao và đảm bảo khả năng bàn giao sản phẩm liên tục thông qua các chu kỳ ngắn. Lộ trình tổng thể kéo dài 16 tuần được chia làm ba giai đoạn chính:

- **Giai đoạn Chuẩn bị (Tuần 1–4):** Tập trung vào việc nghiên cứu thị trường, xác định chiến lược kinh doanh (BRD), chi tiết hóa các yêu cầu chức năng và phi chức năng (SRS) và thiết kế kiến trúc hệ thống tổng quát (SDD).
- **Giai đoạn Phát triển (Tuần 5–12):** Thực hiện phát triển thông qua 3 chu kỳ Sprints chính:
  - *Sprint 1:* Hoàn thiện hệ thống xác thực (Auth), quản lý cấu trúc tòa nhà, tầng, phòng và thiết lập biểu phí dịch vụ.
  - *Sprint 2:* Triển khai module hợp đồng, cư dân và xây dựng quy trình ghi nhận chỉ số điện nước thông qua ảnh chụp.
  - *Sprint 3:* Phát triển bộ máy tính toán hóa đơn, tích hợp thanh toán QR và hoàn thiện giao diện Dashboard.
- **Giai đoạn Kiểm thử và Hoàn thiện (Tuần 13–16):** Song song với quá trình phát triển, các hoạt động QA/QC được thực hiện để đảm bảo chất lượng, thực hiện kiểm thử chấp nhận người dùng (UAT) trước khi đóng gói và bàn giao.

**Công cụ và công nghệ sử dụng:** Để đáp ứng yêu cầu về hiệu suất cao và khả năng bảo trì, hệ sinh thái công nghệ của đề tài được lựa chọn dựa trên các tiêu chí hiện đại và tối ưu:

- **Quản lý dự án và mã nguồn:** Sử dụng mô hình Monorepo với Turborepo và pnpm để quản lý toàn bộ mã nguồn trên GitHub. Lộ trình và tài liệu chuyên môn được quản lý tập trung trên Notion.
- **Công nghệ Backend:** Ngôn ngữ lập trình Rust cùng framework Loco.rs (dựa trên Axum [24]) giúp đảm bảo hiệu suất cao và an toàn bộ nhớ [10]. SeaORM [22] được sử dụng để tương tác với cơ sở dữ liệu PostgreSQL [6].
- **Công nghệ Frontend:** Framework Next.js 16 kết hợp với TailwindCSS v4 cho phiên bản Web Dashboard của chủ nhà, đảm bảo giao diện hiện đại và tốc độ phản hồi nhanh.

- **Ứng dụng di động:** Sử dụng Expo 55 và React Native cho phiên bản Mobile dành cho người thuê, giúp tối ưu hóa thời gian phát triển đa nền tảng (iOS/Android) với giao diện linh hoạt từ NativeWind.
- **Hạ tầng và Vận hành:** Hệ thống được triển khai trên nền tảng Docker [8], vận hành trên hạ tầng đám mây AWS [17] (EC2 cho máy chủ và S3 cho lưu trữ tệp tin), tự động hóa CI/CD qua GitHub Actions [5].

## 1.5 Ý nghĩa đồ án

**Ý nghĩa khoa học:** Đồ án không chỉ dừng lại ở việc xây dựng một ứng dụng quản lý đơn thuần mà còn là quá trình nghiên cứu và ứng dụng các giải pháp công nghệ tiên tiến vào thực tiễn:

- **Ứng dụng ngôn ngữ Rust:** Minh chứng cho việc áp dụng một ngôn ngữ lập trình hiện đại, chú trọng vào an toàn bộ nhớ và hiệu suất vào lĩnh vực phần mềm dân dụng, giúp giảm thiểu các lỗi logic và bảo mật thường gặp.
- **Tối ưu hóa kiến trúc hệ thống:** Nghiên cứu và thực thi mô hình Modular Monolith, giúp cân bằng giữa tốc độ phát triển và khả năng bảo trì. Đồng thời, việc áp dụng Multi-tenancy giúp giải quyết bài toán cách ly dữ liệu và tối ưu tài nguyên hạ tầng cho nhiều khách thuê.
- **Số hóa và xử lý hình ảnh:** Đóng góp vào việc nghiên cứu quy trình tự động hóa dựa trên bằng chứng kỹ thuật số (ảnh chụp), tạo nền tảng cho việc tích hợp các mô hình AI/OCR chuyên sâu hơn trong tương lai.

**Ý nghĩa thực tiễn:** Sự ra đời của Nhatroso Platform mang lại những giá trị thiết thực cho các đối tượng trong hệ sinh thái:

- **Đối với chủ cho thuê:** Xây dựng phong cách quản lý chuyên nghiệp, giảm bớt gánh nặng tính toán thủ công và sai sót. Hệ thống báo cáo thời gian thực giúp chủ nhà nắm bắt chính xác dòng tiền, tình trạng lấp đầy và hiệu quả kinh doanh của từng tòa nhà.
- **Đối với người thuê phòng:** Cung cấp trải nghiệm minh bạch nhất về chi phí. Người thuê có thể dễ dàng quản lý hóa đơn, lịch sử thanh toán và thực hiện các giao dịch không tiền mặt chỉ với vài thao tác trên di động, từ đó gia tăng sự tin tưởng và gắn kết với chủ nhà.
- **Đối với cộng đồng và xã hội:** Góp phần vào mục tiêu chuyển đổi số quốc gia, đặc biệt là trong phân khúc kinh tế hộ gia đình và kinh doanh nhà trọ nhỏ lẻ. Đồ án mở ra hướng đi cho việc hiện đại hóa các dịch vụ dân sinh bằng công nghệ số.

## 1.6 Bố cục đồ án

Đồ án được tổ chức thành bảy chương:

- Chương 1: Giới thiệu — Bối cảnh, mục tiêu và phạm vi đề tài.
- Chương 2: Cơ sở lý thuyết — Kiến thức nền tảng, công nghệ sử dụng và kiến trúc hệ thống.
- Chương 3: Phân tích yêu cầu hệ thống — Stakeholder, yêu cầu chức năng và quản lý nghiệp vụ.
- Chương 4: Thiết kế hệ thống — Mô hình dữ liệu, thiết kế API và quy trình nghiệp vụ.
- Chương 5: Cài đặt và triển khai — Công nghệ, cấu trúc module và triển khai Docker.
- Chương 6: Thử nghiệm và Đánh giá — Chiến lược, kịch bản và kết quả kiểm thử thực tế.
- Chương 7: Kết luận — Tổng kết kết quả đạt được, hạn chế và hướng phát triển tương lai.

# Chương 2

# Cơ sở lý thuyết

Chương này trình bày các khái niệm nền tảng về quản lý, mô hình kiến trúc, các công nghệ chủ chốt và cơ sở thuật toán được áp dụng trong quá trình xây dựng và phát triển hệ thống Nhatroso Platform.

## 2.1 Khái niệm liên quan

### 2.1.1 Hệ thống quản lý (Management System)

Hệ thống quản lý là một tập hợp các quy trình, công cụ và dữ liệu được tổ chức nhằm tối ưu hóa việc điều hành và giám sát một lĩnh vực cụ thể. Trong ngữ cảnh bất động sản, hệ thống quản lý giúp số hóa các thực thể (phòng, cư dân) và tự động hóa các luồng nghiệp vụ (billing, contract management) nhằm giảm tải thao tác thủ công và hạn chế sai sót.

### 2.1.2 Kiến trúc đa khách thuê (Multi-tenancy)

Multi-tenancy[1] [1] là mô hình kiến trúc phần mềm trong đó một phiên bản duy nhất (single instance) của ứng dụng phục vụ nhiều khách hàng hoặc nhóm người dùng khác nhau (tenants). Mỗi khách thuê được cung cấp một không gian làm việc ảo với dữ liệu và cấu hình được cách ly hoàn toàn, mặc dù họ đang chia sẻ chung hạ tầng phần cứng và tài nguyên tính toán.

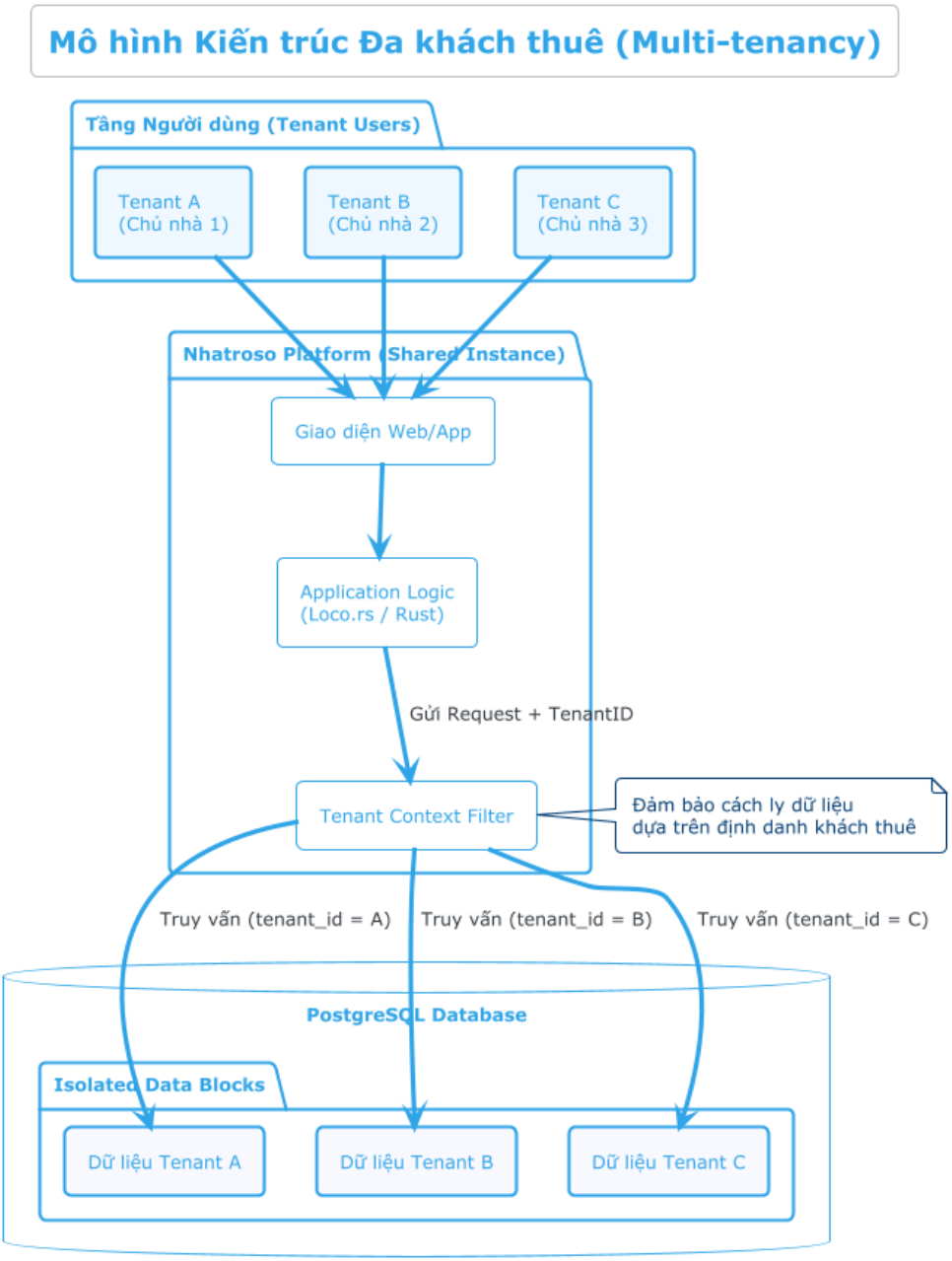
Trong kiến trúc đa khách thuê, việc lựa chọn mô hình cách ly dữ liệu đóng vai trò then chốt:

- **Cơ sở dữ liệu riêng (Database-per-tenant):** Mỗi khách thuê sở hữu một cơ sở dữ liệu vật lý riêng biệt. Đây là mức độ cách ly cao nhất, đảm bảo tính bảo mật và hiệu suất nhưng khó khăn trong việc mở rộng và bảo trì.
- **Schema riêng (Schema-per-tenant):** Các khách thuê chia sẻ chung một cơ sở dữ liệu

[1] Kiến trúc đa khách thuê là một mô hình trong đó một thực thể vật lý của phần mềm phục vụ nhiều nhóm người dùng khác nhau, giúp tối ưu hóa tài nguyên và chi phí vận hành.

nhưng sử dụng các schema (không gian tên) khác nhau. Mô hình này cân bằng giữa tính cách ly và khả năng quản lý.

- **Dùng chung cơ sở dữ liệu (Shared Database, Shared Schema):** Toàn bộ khách thuê lưu trữ dữ liệu trong cùng một tập hợp các bảng, được phân biệt thông qua một cột định danh (*Tenant ID*). Đây là mô hình tối ưu về chi phí và tài nguyên nhất nhưng yêu cầu kiểm soát chặt chẽ ở tầng phần mềm để tránh rò rỉ dữ liệu.




**Hình 2.1:** Mô hình kiến trúc tổng quát của hệ thống Đa khách thuê

### 2.1.3 Số hóa quy trình quản lý nhà trọ

Quy trình quản lý nhà trọ truyền thống thường đối mặt với nhiều thách thức như: sai sót trong tính toán thủ công, mất mát dữ liệu giấy tờ và thiếu tính minh bạch trong đối soát dịch vụ. Nhatroso Platform giải quyết các vấn đề này bằng cách số hóa toàn bộ các điểm chạm (touchpoints) trong vòng đời thuê trọ, tạo ra một hệ thống dữ liệu tập trung và thông suốt.

#### Số hóa hợp đồng thuê trọ

Hợp đồng thuê trọ trực tuyến được xây dựng dựa trên các biểu mẫu chuẩn pháp lý Việt Nam, cho phép lưu trữ dưới dạng dữ liệu có cấu trúc thay vì văn bản thuần túy. Việc số hóa hợp đồng giúp hệ thống tự động trích xuất các thông tin quan trọng như: giá thuê, kỳ hạn thanh toán, tiền đặt cọc và danh sách cư dân. Đây là dữ liệu đầu vào quan trọng để kích hoạt các luồng nghiệp vụ tự động tiếp theo như tính toán hóa đơn và quản lý kỳ hạn.

#### Tự động hóa quản lý chỉ số điện nước

Đây là khâu dễ phát sinh tranh chấp nhất trong quản lý nhà trọ. Nhatroso Platform số hóa quy trình này thông qua mô hình tương tác giữa Chủ nhà và Người thuê:

- Hệ thống tự động gửi yêu cầu báo số định kỳ tới ứng dụng di động của người thuê.
- Người thuê thực hiện chụp ảnh đồng hồ và gửi trực tiếp qua ứng dụng. Hình ảnh này phục vụ như bằng chứng vật lý, giúp tăng tính minh bạch.
- Dữ liệu sau khi chủ nhà phê duyệt sẽ được lưu trữ kèm ảnh chụp, làm cơ sở để tính toán chi phí tiêu thụ chính xác dựa trên các cấu hình giá bậc thang hoặc giá cố định.

#### Phát hành hóa đơn và Đối soát tự động

Hệ thống tự động tổng hợp tất cả các khoản chi phí bao gồm: tiền thuê phòng (từ hợp đồng), phí dịch vụ cố định (vệ sinh, internet) và phí biến đổi (điện, nước dựa trên chỉ số). Hóa đơn số được phát hành với mã định danh duy nhất và gửi thông báo tức thời tới người thuê.

Mỗi hóa đơn trải qua một vòng đời trạng thái được kiểm soát chặt chẽ: *Chưa thanh toán (Unpaid)* $\rightarrow$ *Đang xác nhận (Pending)* $\rightarrow$ *Đã thanh toán (Paid)* hoặc *Huỷ (Voided)*. Toàn bộ quá trình thay đổi trạng thái đều được ghi lại trong nhật ký hệ thống (Audit trail), đảm bảo khả năng truy xuất nguồn gốc và minh bạch tài chính cho cả hai bên.

## 2.2 Công nghệ sử dụng

### 2.2.1 Ngôn ngữ lập trình Rust

Rust là ngôn ngữ lập trình hệ thống hiện đại, tập trung vào ba yếu tố cốt lõi: hiệu suất, sự an toàn và tính song song. Với cơ chế *Ownership*[2] (Quyền sở hữu) và *Borrow Checker* [10], Rust giải quyết triệt để các lỗi về truy cập bộ nhớ phổ biến như null pointers, buffer overflows hay data races mà không cần đến bộ dọn rác (Garbage Collector).

**Lý do lựa chọn:**

- **Độ tin cậy tuyệt đối:** Với một hệ thống quản lý tài chính và hợp đồng như Nhatroso, tính toàn vẹn của dữ liệu và sự ổn định của hệ thống là ưu tiên hàng đầu. Rust đảm bảo an toàn bộ nhớ ở mức biên dịch, giúp giảm thiểu rủi ro lỗi runtime.
- **Hiệu suất tối ưu:** Tốc độ thực thi của Rust tương đương với C++, cho phép hệ thống xử lý hàng ngàn yêu cầu đồng thời với mức tiêu thụ tài nguyên cực thấp, giúp tối ưu chi phí vận hành trên hạ tầng đám mây.
- **Hệ sinh thái hiện đại:** Công cụ quản lý gói Cargo và hệ thống kiểu dữ liệu mạnh mẽ giúp việc phát triển các module nghiệp vụ phức tạp trở nên khoa học và ít lỗi hơn.

### 2.2.2 Framework Loco.rs

Loco là framework phát triển web theo triết lý "One-person Framework" dành cho Rust, được lấy cảm hứng mạnh mẽ từ Ruby on Rails [12]. Framework này cung cấp đầy đủ các thành phần cần thiết để xây dựng một ứng dụng web hoàn chỉnh từ CLI scaffolding, database migration đến hệ thống background jobs và mailer. Theo triết lý "Convention over Configuration"[3], Loco giúp đồng bộ hóa cấu trúc dự án một cách nhanh chóng.

**Lý do lựa chọn:**

- **Nâng cao năng suất phát triển:** Rust thường bị xem là có tốc độ phát triển chậm do tính phức tạp. Loco.rs giải quyết vấn đề này bằng cách đưa ra các quy tắc "Convention over Configuration" (Quy ước hơn cấu hình), giúp lập trình viên tập trung vào logic nghiệp vụ thay vì các cấu trúc hạ tầng API.
- **Kiến trúc Modular:** Loco khuyến khích cách tổ chức mã nguồn theo hướng module hóa, hoàn toàn phù hợp với kiến trúc Modular Monolith mà đề tài hướng tới.

[2]Quyền sở hữu (Ownership) là tập hợp các quy tắc quản lý bộ nhớ của Rust mà không cần bộ dọn rác, đảm bảo an toàn bộ nhớ ngay tại thời điểm biên dịch.

[3]Quy ước hơn cấu hình là một mô hình thiết kế phần mềm nhằm giảm thiểu số lượng quyết định mà lập lập trình viên phải thực hiện mà không làm mất đi tính linh hoạt.

- **Tích hợp sẵn các công cụ mạnh:** Việc tích hợp sẵn Axum (HTTP engine) và SeaORM giúp việc xây dựng các REST API bất đồng bộ (async) trở nên dễ dàng và chuẩn hóa.

### 2.2.3 Next.js 16

Next.js 16 là nền tảng React hiện đại nhất hiện nay, tiên phong trong việc hỗ trợ *React Server Components*[4] (RSC) và *App Router*, giúp tối ưu hóa hiệu năng render phía máy chủ và cải thiện đáng kể tốc độ tải trang đầu tiên (FCP) [25].

**Lý do lựa chọn:**

- **Trải nghiệm người dùng mượt mà:** Với khả năng Server-side Rendering (SSR) và Static Site Generation (SSG), hệ thống Dashboard dành cho chủ nhà có tốc độ phản hồi nhanh, mượt mà như một ứng dụng desktop.
- **Tích hợp full-stack:** Next.js cung cấp các API Routes và Middleware mạnh mẽ, cho phép xử lý các tác vụ như Internationalization (i18n) và xác thực JWT một cách đồng bộ từ backend đến frontend.
- **Cộng đồng và công cụ hỗ trợ:** Hệ sinh thái TailwindCSS v4 và các thư viện UI như Flowbite giúp việc xây dựng các dashboard chuyên nghiệp, trực quan trở nên nhanh chóng và chuẩn mực.

### 2.2.4 Expo và React Native

Expo là một hệ sinh thái mã nguồn mở mạnh mẽ giúp đơn giản hóa quá trình phát triển ứng dụng di động đa nền tảng bằng JavaScript/TypeScript [20]. React Native cho phép xây dựng ứng dụng di động với hiệu năng gần như bản địa (native performance).

**Lý do lựa chọn:**

- **Phát triển đa nền tảng (Cross-platform):** Chỉ với một codebase duy nhất, hệ thống có thể triển khai đồng thời trên cả iOS và Android, giúp tiết kiệm 50% nguồn lực và thời gian phát triển so với lập trình native truyền thống.
- **Chia sẻ mã nguồn (Code sharing):** Việc sử dụng cùng ngôn ngữ TypeScript với Frontend Web cho phép chia sẻ các kiểu dữ liệu, logic xử lý và các bản dịch đa ngôn ngữ, giảm thiểu sai lệch giữa các nền tảng.
- **Ecosystem ổn định:** Expo cung cấp các module sẵn có để tương tác với phần cứng như Camera (quét đồng hồ điện), Secure Store (lưu trữ token) và hệ thống thông báo Push Notifications, giúp đẩy nhanh tiến độ hoàn thiện app.

[4] RSC là các thành phần React được thực thi hoàn toàn trên máy chủ, giúp giảm kích thước gói tin JavaScript gửi đến trình duyệt và cải thiện hiệu năng.

### 2.2.5 Cơ sở dữ liệu PostgreSQL và SeaORM

Hệ thống sử dụng PostgreSQL [6] làm cơ sở dữ liệu quan hệ chính nhờ tính tin cậy tuyệt đối, khả năng xử lý giao dịch (ACID) mạnh mẽ và cộng đồng hỗ trợ rộng lớn. SeaORM là thư viện ORM bất đồng bộ được xây dựng riêng cho Rust [22].

**Lý do lựa chọn:**

- **Tính toàn vẹn dữ liệu:** PostgreSQL hỗ trợ các ràng buộc (constraints) và kiểu dữ liệu phức tạp, đảm bảo các quan hệ giữa Phòng - HĐ - Hóa đơn luôn chính xác.
- **Lập trình kiểu mạnh (Type-safe ORM):** SeaORM ánh xạ dữ liệu trực tiếp vào các structs trong Rust ngay khi biên dịch, giúp phát hiện sớm các lỗi sai lệch schema trước khi triển khai.
- **Tính bất đồng bộ (Async):** Kết hợp với Rust async runtime, SeaORM cho phép xử lý hàng ngàn kết nối cơ sở dữ liệu đồng thời mà không gây nghẽn tiến trình, phù hợp cho quy trình billing quy mô lớn.

## 2.3 Mô hình và Thuật toán

### 2.3.1 Kỹ thuật nhận dạng ký tự quang học (OCR) với AI

Nhận dạng ký tự quang học (OCR - Optical Character Recognition) là quy trình chuyển đổi hình ảnh chứa văn bản thành dữ liệu văn bản có cấu trúc mà máy tính có thể xử lý. Trong bối cảnh quản lý nhà trọ, việc trích xuất chỉ số từ ảnh chụp đồng hồ điện, nước là một bài toán thách thức do sự đa dạng về chủng loại đồng hồ, điều kiện ánh sáng kém và góc chụp không cố định.

Hệ thống Nhatroso Platform lựa chọn giải pháp **Google Cloud Vision API**[5] [3] — một dịch vụ cung cấp mô hình học máy dưới dạng giao diện lập trình (Model-as-a-Service) — thay vì tự huấn luyện một mô hình học máy riêng biệt. Các lý do chủ chốt cho quyết định này bao gồm:

- **Tối ưu hóa nguồn lực và thời gian:** Việc huấn luyện một mô hình học máy (như CNN hay Transformer) cho bài toán OCR yêu cầu một tập dữ liệu (dataset) khổng lồ gồm hàng chục ngàn ảnh chụp đồng hồ đã được gán nhãn chính xác. Sử dụng API có sẵn giúp đội ngũ phát triển bỏ qua giai đoạn thu thập dữ liệu và huấn luyện phức tạp, tập trung hoàn toàn vào việc xây dựng luồng nghiệp vụ.
- **Độ chính xác và tính bền vững (Robustness):** Google Cloud Vision API ứng dụng các

[5] Dịch vụ thị giác máy tính dựa trên nền tảng đám mây của Google, cung cấp các mô hình học máy được huấn luyện sẵn để nhận diện vật thể và văn bản.

mô hình học sâu (Deep Learning) tiên tiến nhất, được huấn luyện trên quy mô dữ liệu toàn cầu. Nhờ đó, hệ thống có khả năng xử lý tốt các tình huống khó khăn như: góc chụp bị nghiêng (perspective distortion), bề mặt kính đồng hồ bị lóa hoặc trầy xước, và các điều kiện ánh sáng yếu.

- **Khả năng tự động tối ưu hóa:** Google liên tục cập nhật và cải thiện các thuật toán nhận dạng mà không yêu cầu nhà phát triển phải can thiệp. Việc sử dụng dịch vụ đám mây còn giúp hệ thống tự động mở rộng (auto-scaling) theo lượng tải thực tế mà không cần quản lý hạ tầng phần cứng (GPU/CPU).
- **Mô hình chi phí hiệu quả:** Với mô hình trả phí theo lượt sử dụng (Pay-as-you-go), dự án tránh được các chi phí đầu tư ban đầu cho máy chủ huấn luyện và giảm thiểu rủi ro vận hành.

Quy trình OCR trong hệ thống được thực hiện qua hai giai đoạn tự động:

1. **Phát hiện văn bản (Text Detection):** Xác định tọa độ (bounding boxes) của các dãy số trên mặt đồng hồ.
2. **Nhận dạng ký tự (Text Recognition):** Chuyển đổi các vùng ảnh đã phát hiện thành chuỗi số thực tế, phục vụ cho việc tính toán hóa đơn.

Mặc dù trong phiên bản MVP, OCR được hỗ trợ như một công cụ gợi ý để chủ nhà phê duyệt lần cuối, việc nền tảng hóa quy trình này đặt cơ sở vững chắc cho khả năng tự động hóa hoàn toàn trong các giai đoạn phát triển tiếp theo của Nhatroso Platform.

### 2.3.2 Internet of Things (IoT) và Giao thức MQTT

IoT là mạng lưới các thiết bị vật lý có khả năng thu thập và trao đổi dữ liệu qua internet. Giao thức MQTT (Message Queuing Telemetry Transport) là phương thức truyền tin nhẹ, dựa trên mô hình Publish/Subscribe, cực kỳ hiệu quả cho các thiết bị IoT có băng thông thấp và tài nguyên hạn chế [14]. Mặc dù Nhatroso Platform hiện tại tập trung vào quy trình thủ công thông qua ảnh chụp, nhưng kiến trúc IoT với giao thức MQTT được xem là nền tảng lý thuyết để phát triển tính năng đọc chỉ số tự động từ các Smart Meter trong tương lai.

## 2.4 Quy trình phát triển phần mềm

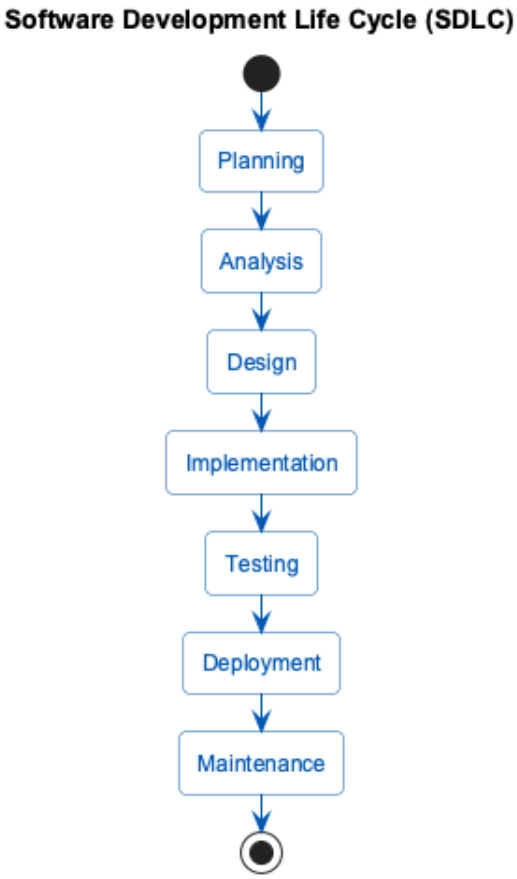
### 2.4.1 Vòng đời phát triển phần mềm (SDLC)

Vòng đời phát triển phần mềm (The Software Development Life Cycle - SDLC) là một quy trình chuẩn hóa được sử dụng để thiết kế, xây dựng và kiểm thử phần mềm có chất lượng

cao [19]. Mục tiêu của SDLC là cung cấp một sản phẩm đáp ứng đầy đủ yêu cầu của người dùng, hoàn thành trong thời gian và ngân sách dự kiến.

Một chu trình SDLC tiêu chuẩn thường bao gồm các giai đoạn chính sau:

- **Lập kế hoạch (Planning):** Đây là giai đoạn khởi đầu nhằm xác định tầm nhìn và tính khả thi của dự án. Các hoạt động chính bao gồm: định nghĩa mục tiêu cốt lõi, xác định phạm vi công việc (scope), ước lượng nguồn lực (nhân sự, thời gian, ngân sách) và đánh giá các rủi ro tiềm ẩn. Kết quả của giai đoạn này là bản kế hoạch tổng thể giúp định hướng cho toàn bộ quá trình phát triển.
- **Phân tích yêu cầu (Analysis):** Giai đoạn này tập trung vào việc hiểu rõ nhu cầu của các bên liên quan (stakeholders). Các kỹ sư phần mềm thực hiện thu thập, phân loại và chi tiết hóa các yêu cầu chức năng (functional requirements) và yêu cầu phi chức năng (non-functional requirements). Sản phẩm quan trọng nhất là Tài liệu Đặc tả yêu cầu phần mềm (SRS), đóng vai trò là "kim chỉ nam"cho việc thiết kế và kiểm thử.
- **Thiết kế (Design):** Dựa trên tài liệu SRS, giai đoạn thiết kế sẽ cụ thể hóa cấu trúc kỹ thuật của hệ thống. Các hoạt động bao gồm: thiết kế kiến trúc tổng thể (Architecture Design), thiết kế cơ sở dữ liệu (Database Schema), thiết kế giao diện (UI/UX Design) và các sơ đồ logic nghiệp vụ. Giai đoạn này giúp chuyển đổi các yêu cầu trừu tượng thành các bản vẽ kỹ thuật chi tiết.
- **Cài đặt (Implementation):** Đây là giai đoạn trực tiếp tạo ra sản phẩm thông qua việc lập trình (coding). Các lập trình viên sử dụng các ngôn ngữ và công nghệ đã chọn để thực hiện các module tính năng dựa trên bản thiết kế. Việc tuân thủ các tiêu chuẩn lập trình (coding standards) và sử dụng hệ thống quản lý phiên bản (Git) là yêu cầu bắt buộc để đảm bảo chất lượng mã nguồn.
- **Kiểm thử (Testing):** Sau khi cài đặt, phần mềm phải trải qua quy trình kiểm thử nghiêm ngặt để phát hiện và khắc phục lỗi (bugs). Quy trình này bao gồm nhiều cấp độ: kiểm thử đơn vị (Unit Test) cho các hàm/lớp riêng lẻ, kiểm thử tích hợp (Integration Test) cho sự tương tác giữa các module, và kiểm thử hệ thống (System Test) trên toàn bộ ứng dụng để đảm bảo đáp ứng đúng các yêu cầu ban đầu.
- **Triển khai và Bảo trì (Deployment & Maintenance):** Khi phần mềm đạt yêu cầu chất lượng, nó sẽ được triển khai lên môi trường sản xuất (Production). Sau khi phát hành, hệ thống cần được giám sát, bảo trì định kỳ để khắc phục các vấn đề phát sinh và cập nhật các tính năng mới theo phản hồi của người dùng thực tế.




**Hình 2.2:** Các giai đoạn trong vòng đời phát triển phần mềm (SDLC)

### 2.4.2 Mô hình Agile Scrum

Agile là một triết lý phát triển phần mềm tập trung vào việc chuyển giao giá trị nhanh chóng thông qua các chu kỳ lặp và sự cộng tác liên tục. Scrum là một khung làm việc (framework) cho phép các nhóm giải quyết những vấn đề phức tạp, trong khi vẫn cung cấp các sản phẩm có giá trị cao nhất một cách sáng tạo [16]. Scrum dựa trên lý thuyết quản lý tiến trình thực nghiệm (Empiricism) với ba trụ cột chính: *Sự minh bạch (Transparency)*, *Sự thanh tra (Inspection)* và *Sự thích nghi (Adaptation)*.

**Các vai trò chính trong Scrum (Scrum Roles):**

- **Product Owner (Người sở hữu sản phẩm):** Là người chịu trách nhiệm tối đa hóa giá trị của sản phẩm thông qua việc quản lý Danh mục sản phẩm (Product Backlog). PO là người đưa ra các quyết định về thứ tự ưu tiên của các tính năng dựa trên nhu cầu của khách hàng và lợi ích kinh doanh.
- **Scrum Master (Người điều phối):** Là người chịu trách nhiệm đảm bảo nhóm hiểu và vận hành đúng theo quy trình Scrum. Scrum Master đóng vai trò là một "người phục vụ"(servant-leader), giúp loại bỏ các rào cản cản trở tiến độ của nhóm và huấn luyện

nhóm cải tiến năng suất.

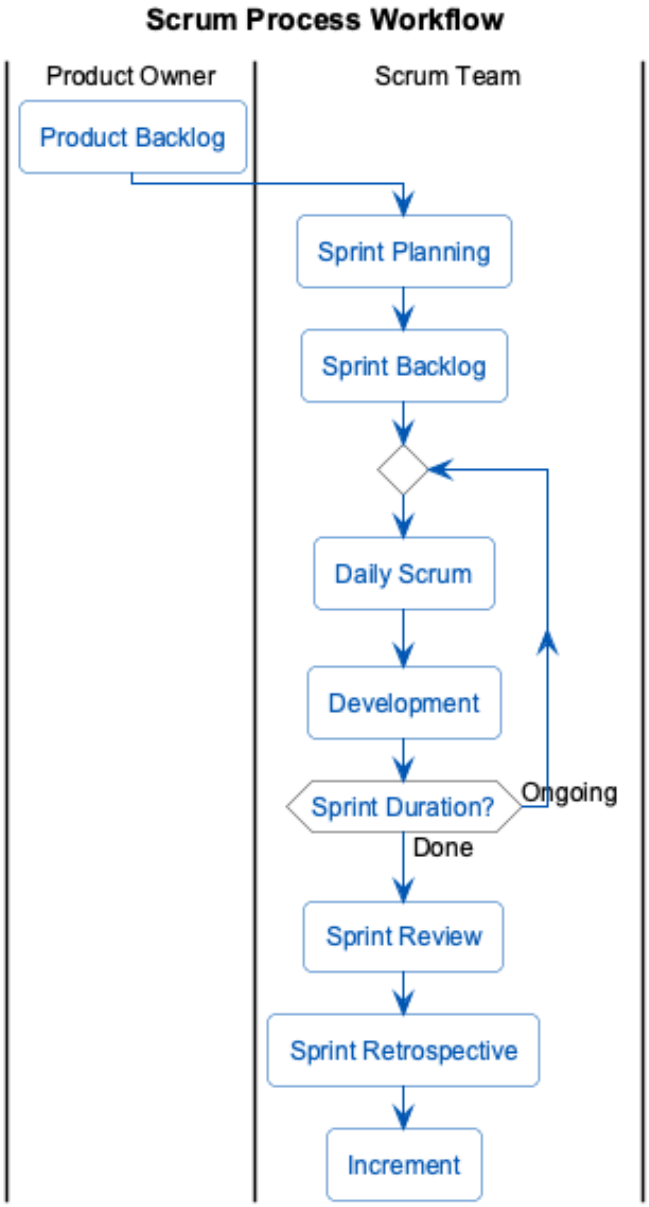
- **Development Team (Nhóm phát triển):** Là một nhóm liên chức năng (Cross-functional) và tự quản lý (Self-organizing), bao gồm các chuyên gia trực tiếp thực hiện công việc để tạo ra các phần tăng trưởng sản phẩm (Increments) có khả năng phát hành ở cuối mỗi Sprint.

**Các sự kiện trong Scrum (Scrum Events):**

- **Sprint (Chu kỳ phát triển):** Là một đơn vị thời gian cố định (thường từ 1-4 tuần) trong đó một phiên bản sản phẩm có thể chuyển giao được tạo ra. Toàn bộ dự án là một chuỗi các Sprint liên tiếp.
- **Sprint Planning (Lập kế hoạch Sprint):** Buổi họp diễn ra ở đầu mỗi Sprint để xác định mục tiêu của Sprint đó và lựa chọn các hạng mục từ Product Backlog để thực hiện.
- **Daily Scrum (Họp hằng ngày):** Cuộc họp ngắn 15 phút hằng ngày để nhóm đồng bộ hóa công việc, chia sẻ những gì đã làm, những gì sẽ làm và các khó khăn đang gặp phải.
- **Sprint Review (Sơ kết Sprint):** Diễn ra ở cuối Sprint để kiểm tra kết quả công việc (Increment) và xác định các công việc tiếp theo.
- **Sprint Retrospective (Cải tiến Sprint):** Là dịp để nhóm nhìn lại quá trình làm việc, tìm ra các điểm tốt và các điểm cần cải thiện về mặt quy trình, nhân sự và công cụ để áp dụng cho Sprint sau.

**Các tạo tác trong Scrum (Scrum Artifacts):**

- **Product Backlog:** Danh sách ưu tiên các tính năng, yêu cầu cần có cho sản phẩm. Đây là nguồn duy nhất cho mọi thay đổi của sản phẩm.
- **Sprint Backlog:** Tập hợp các hạng mục từ Product Backlog được chọn cho Sprint hiện tại cùng với kế hoạch chi tiết để hoàn thành mục tiêu Sprint.
- **Increment (Phần tăng trưởng sản phẩm):** Là kết quả của tất cả các hạng mục Product Backlog đã hoàn thành trong Sprint và giá trị của các increment từ những Sprint trước đó.




**Hình 2.3:** Cấu trúc quy trình thực thi theo mô hình Agile Scrum

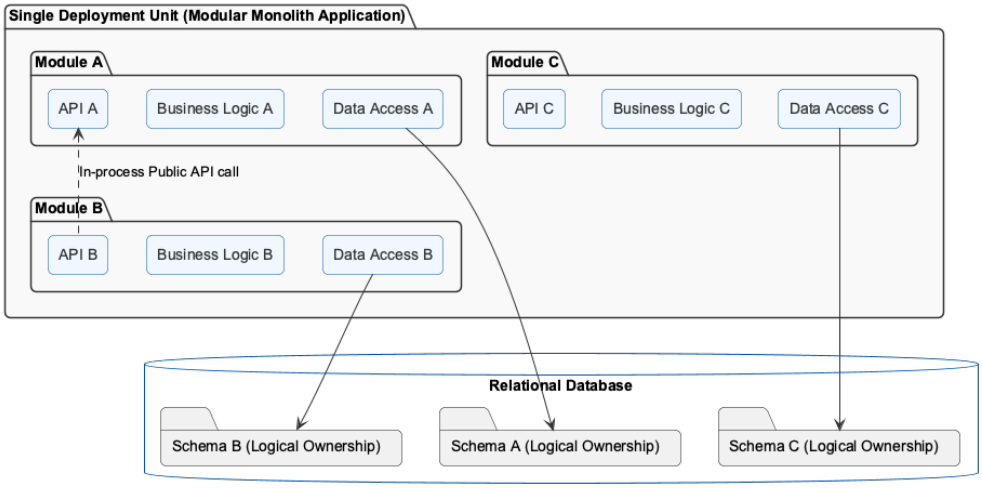
## 2.5 Kiến trúc hệ thống và Bảo mật

### 2.5.1 Kiến trúc Modular Monolith

Kiến trúc Modular Monolith là sự kết hợp giữa tính đơn giản trong triển khai của kiến trúc Monolith truyền thống và tính tách biệt logic của kiến trúc Microservices [7]. Trong mô hình này, toàn bộ ứng dụng được đóng gói và triển khai dưới dạng một đơn vị duy nhất (Single Deployment Unit), nhưng bên trong mã nguồn được chia thành các module chức năng độc lập và có ranh giới rõ ràng.

**Lợi ích và Lý do lựa chọn:**

- **Độ phức tạp thấp:** Khác với Microservices, Modular Monolith không yêu cầu quản lý mạng phức tạp, độ trễ truyền tin hay các bài toán về tính nhất quán dữ liệu phân tán.
- **Tách biệt logic (Logical Isolation):** Mỗi module (như User, Billing, Property) sở hữu logic và dữ liệu riêng biệt, chỉ giao tiếp với nhau qua các giao thức công khai (Public APIs). Điều này giúp giảm thiểu sự phụ thuộc lẫn nhau (coupling) và tăng khả năng bảo trì.
- **Khả năng mở rộng bền vững:** Nhờ cấu trúc module hóa, hệ thống có thể dễ dàng được tách thành các Microservices riêng lẻ trong tương lai nếu nhu cầu về hiệu năng và mở rộng tăng cao mà không cần phải viết lại toàn bộ mã nguồn.




**Hình 2.4:** Mô hình minh họa kiến trúc Modular Monolith

Trong đồ án này, kiến trúc Modular Monolith được triển khai thông qua các module logic trong Rust, giúp đội ngũ phát triển quản lý mã nguồn một cách khoa học, dễ dàng thực hiện kiểm thử đơn vị và bảo đảm tính an toàn cho từng thành phần của Nhatroso Platform.

JWT là một tiêu chuẩn mở (RFC 7519) [9] bao gồm ba phần: Header (thuật toán mã hóa), Payload (chứa các claims như user_id, role) và Signature (chữ ký số để đảm bảo tính toàn vẹn). Trong hệ thống, JWT đóng vai trò là phương thức xác thực chính, cho phép Mobile App và Web Dashboard gửi các yêu cầu an toàn tới Backend mà không cần lưu trữ session trên server.

### 2.5.2 Kiến trúc Phân quyền RBAC

Kiểm soát truy cập dựa trên vai trò (Role-Based Access Control - RBAC) là một phương pháp quản lý quyền truy cập hệ thống dựa trên chức năng và nhiệm vụ của từng người dùng trong tổ chức [4]. Thay vì gán trực tiếp quyền hạn cho từng cá nhân, RBAC sử dụng thực thể

trung gian là *Vai trò (Role)*. Mô hình này tuân thủ nghiêm ngặt nguyên tắc Quyền hạn tối thiểu (Principle of Least Privilege), đảm bảo người dùng chỉ có đủ các quyền cần thiết để thực hiện công việc của mình.

Cấu trúc RBAC được xây dựng trên ba thành phần nền tảng:

1. **Người dùng (User):** Các cá nhân hoặc hệ thống tương tác với nền tảng.
2. **Vai trò (Role):** Một tập hợp các quyền hạn được định nghĩa sẵn dựa trên chức năng nghiệp vụ.
3. **Quyền hạn (Permission):** Khả năng thực hiện các thao tác cụ thể (Xem, Tạo, Sửa, Xóa) trên một tài nguyên nhất định.

Trong hệ thống Nhatroso Platform, cơ chế RBAC phân tách rõ rệt hai nhóm vai trò chủ chốt:

- **Owner (Chủ nhà):** Đóng vai trò là quản trị viên của không gian khách thuê (Tenant space). Vai trò này có toàn quyền (Full Access) đối với các tài nguyên thuộc sở hữu của mình, bao gồm: quản lý cấu trúc tòa nhà, thiết lập đơn giá dịch vụ, phê duyệt hợp đồng, khởi tạo hóa đơn và xem báo cáo tài chính tổng thể.
- **Tenant (Người thuê):** Đóng vai trò là người sử dụng dịch vụ cuối cùng. Vai trò này bị giới hạn quyền truy cập theo cơ chế cách ly dữ liệu cá nhân. Người thuê chỉ có quyền xem thông tin hợp đồng và hóa đơn của chính mình, gửi yêu cầu báo số điện nước và nhận các thông báo từ hệ thống.

Về mặt kỹ thuật, cơ chế RBAC được thực thi thông qua mã định danh vai trò (Role Claim) được nhúng bên trong mã thông báo JWT. Khi người dùng gửi yêu cầu tới API, một tầng phần mềm trung gian (*RBAC Middleware*) sẽ thực hiện giải mã JWT, kiểm tra vai trò của người dùng và đối chiếu với danh mục quyền hạn của tệp tin API (Endpoint) để quyết định cho phép hoặc từ chối truy cập. Cơ chế này không chỉ giúp bảo mật dữ liệu mà còn giúp việc quản trị và mở rộng các nhóm quyền hạn mới trở nên linh hoạt và tập trung.

## 2.6 Triển khai và Tích hợp liên tục (CI/CD)

Triển khai và Tích hợp liên tục (CI/CD) là một tập hợp các phương pháp và tri thức kỹ thuật nhằm tự động hóa các giai đoạn trong quá trình phát triển phần mềm, từ khi lập trình viên hoàn thành mã nguồn đến khi sản phẩm được vận hành thực tế. Mục tiêu cốt lõi của CI/CD là tăng tốc độ chuyển giao (velocity), nâng cao chất lượng phần mềm thông qua kiểm thử tự động và giảm thiểu rủi ro lỗi do con người trong quá trình triển khai.

### 2.6.1 Tích hợp liên tục (Continuous Integration - CI)

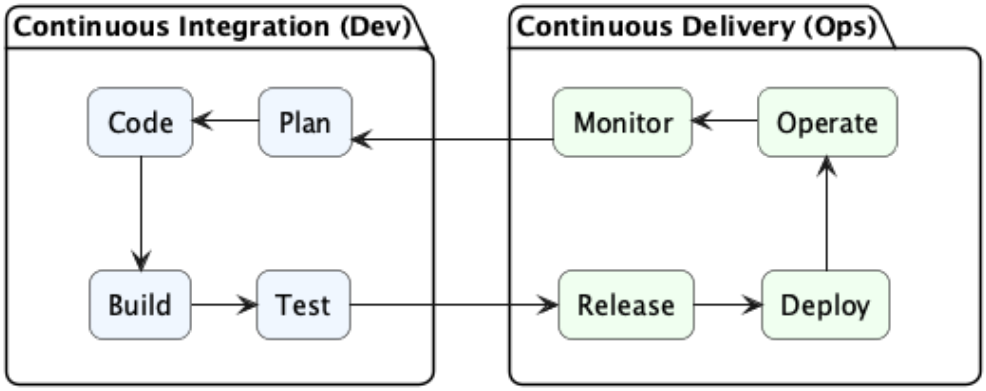
Continuous Integration là giai đoạn tập trung vào việc tự động hóa quá trình xây dựng (build) và kiểm tra mã nguồn ngay khi có các thay đổi mới được đẩy lên hệ thống quản lý phiên bản (Git). Các thành phần kiến thức chính trong CI bao gồm:

- **Tự động hóa xây dựng (Automated Build):** Hệ thống tự động biên dịch mã nguồn để đảm bảo không có lỗi cú pháp hoặc lỗi phụ thuộc. Đối với ngôn ngữ Rust, giai đoạn này được thực thi thông qua công cụ Cargo.
- **Kiểm soát chất lượng mã nguồn (Linting):** Sử dụng các công cụ phân tích mã tĩnh (Static Analysis) như Clippy (Rust) hoặc ESLint (TypeScript) để phát hiện các đoạn mã không tối ưu, vi phạm quy tắc lập trình hoặc tiềm ẩn lỗi bảo mật.
- **Kiểm thử tự động (Automated Testing):** Thực thi đồng thời các bộ kiểm thử đơn vị (Unit Tests) và kiểm thử tích hợp (Integration Tests) để xác nhận rằng các thay đổi mới không làm phá vỡ các chức năng hiện có của hệ thống.

### 2.6.2 Triển khai liên tục (Continuous Delivery/Deployment - CD)

Giai đoạn CD tiếp nối sau khi giai đoạn CI hoàn thành thành công, tập trung vào việc đưa sản phẩm đến môi trường vận hành một cách an toàn và tự động:

- **Đóng gói Container (Containerization):** Sử dụng Docker để đóng gói ứng dụng cùng với toàn bộ môi trường runtime, thư viện phụ thuộc vào một hình ảnh (Image) duy nhất. Điều này đảm bảo tính nhất quán tuyệt đối giữa môi trường phát triển và môi trường sản xuất.
- **Quản lý kho hình ảnh (Registry Management):** Các Docker Image được đẩy lên và lưu trữ tại các kho chứa bảo mật như Amazon ECR (Elastic Container Registry) hoặc Docker Hub, phục vụ cho việc quản lý phiên bản và triển khai.
- **Triển khai không gián đoạn (Zero-downtime Deployment):** Áp dụng các kỹ thuật như Rolling Update để cập nhật phiên bản mới của hệ thống Backend mà không gây ngưng trệ dịch vụ đối với người dùng cuối.




**Hình 2.5:** Mô hình vòng lặp minh họa luồng CI/CD

Trong đồ án Nhatroso Platform, toàn bộ luồng CI/CD được tự động hóa thông qua công cụ GitHub Actions, kết hợp với hạ tầng điện toán đám mây Amazon Web Services (AWS), tạo ra một quy trình phát triển chuyên nghiệp, tin cậy và linh hoạt.

# Chương 3

# Phân tích yêu cầu hệ thống

## 3.1 Phân tích stakeholder

Trong quản lý dự án phần mềm, việc xác định các bên liên quan (Stakeholders) là bước quan trọng để hiểu rõ kỳ vọng và xây dựng chiến lược phù hợp. Dự án Nhatroso ứng dụng Ma trận Quyền hạn - Mối quan tâm[1] [11] để phân loại các nhóm stakeholder cốt lõi.

**Bảng 3.1:** Ma trận phân tích stakeholder (Interest Matrix)

| Nhóm Stakeholder | Quyền lực | Mối quan tâm | Ưu tiên | Chiến lược tiếp cận |
|---|---|---|---|---|
| Người cho thuê (Chủ nhà) | Cao | Rất cao | 1 | Quản lý chặt chẽ (Key Players) |
| Sinh viên phát triển | Cao | Rất cao | 1 | Quản lý chặt chẽ (Key Players) |
| Giảng viên cố vấn | Cao | Trung bình | 2 | Làm hài lòng (Keep Satisfied) |
| Người thuê trọ | Thấp | Cao | 2 | Giữ thông tin (Keep Informed) |

### 3.1.1 Chi tiết các nhóm stakeholder

- **Người cho thuê (Quản lý nhà trọ):** Đây là nhóm người dùng trung tâm, đóng vai trò quyết định trong việc áp dụng hệ thống vào thực tế kinh doanh. Họ quan tâm đến khả năng tự động hóa quy trình quản lý phòng, minh bạch hóa đơn điện nước và báo cáo tài chính chính xác để giảm thiểu rủi ro và tranh chấp với người thuê.
- **Người thuê trọ (Thanh niên/Sinh viên):** Nhóm người dùng trực tiếp sử dụng Mobile App để tương tác với chủ nhà. Mối quan tâm lớn nhất của họ là tính minh bạch trong việc tính toán chỉ số, sự thuận tiện khi gửi dữ liệu và theo dõi lịch sử thanh toán một cách dễ dàng, nhanh chóng.

[1] Một khung quản trị được sử dụng để phân loại các bên liên quan dựa trên mức độ ảnh hưởng và mức độ quan tâm của họ đối với dự án.

- **Giảng viên cố vấn:** Đóng vai trò giám sát về mặt chuyên môn và học thuật. Họ cung cấp định hướng về phương pháp luận, kiểm soát chất lượng kỹ thuật của giải pháp và đánh giá sự tuân thủ các quy chuẩn kỹ thuật trong khuôn khổ đồ án tốt nghiệp.
- **Sinh viên chịu trách nhiệm dự án:** Là người trực tiếp thiết kế, triển khai và vận hành hệ thống. Người này có quyền tự quyết về mặt kiến trúc và công nghệ, đồng thời có mối quan tâm cao nhất đến sự thành công, tính ổn định và khả năng mở rộng của nền tảng Nhatroso.

## 3.2 Yêu cầu chức năng

Dựa trên thực tế nghiệp vụ quản lý nhà trọ và các thành phần kỹ thuật đã được triển khai trong mã nguồn dự án Nhatroso Platform, các yêu cầu chức năng cốt lõi được hệ thống hóa thành sáu nhóm chính dưới đây.

### 3.2.1 Quản lý người dùng và phân quyền (Authentication & RBAC)

Hệ thống cung cấp cơ chế bảo mật và phân tách vai trò rõ ràng để đảm bảo tính riêng tư của dữ liệu:

- **Đăng ký và Đăng nhập:** Hỗ trợ xác thực qua Email và Số điện thoại với cơ chế băm mật khẩu Argon2 [2]. Hệ thống sử dụng JWT (JSON Web Token)[2] [9] để duy trì phiên làm việc cho cả Web Dashboard và Mobile App.
- **Phân quyền dựa trên vai trò (RBAC):** Phân chia người dùng thành hai nhóm chính: Chủ nhà (`Owner`) có toàn quyền quản lý tài nguyên và Người thuê (`Tenant`) có quyền truy cập hạn chế vào thông tin hóa đơn và phòng ở của cá nhân.
- **Quản lý hồ sơ:** Cho phép người dùng cập nhật thông tin cá nhân và lưu trữ an toàn các tài liệu định danh (căn cước công dân) phục vụ cho việc lập hợp đồng.

### 3.2.2 Quản lý hạ tầng bất động sản (Property Management)

Module này giúp số hóa toàn bộ cấu trúc vật lý của cơ sở cho thuê:

- **Quản lý phân cấp tòa nhà:** Hỗ trợ cấu hình sơ đồ theo cấu trúc: Tòa nhà → Tầng → Phòng. Mỗi thực thể đều có mã định danh và thuộc tính riêng biệt.
- **Theo dõi trạng thái phòng:** Mỗi phòng có trạng thái thực tế (`Empty`, `Occupied`, `Reserved`, `Maintenance`) được cập nhật tự động theo vòng đời hợp đồng hoặc thao tác

[2] Một tiêu chuẩn mã nguồn mở dùng để tạo các mã xác thực phi trạng thái, giúp truyền tải thông tin giữa các bên dưới dạng đối tượng JSON một cách an toàn.

thủ công của chủ nhà.

### 3.2.3 Quản lý dịch vụ và bảng giá

Thành phần này đảm bảo tính linh hoạt trong việc cấu hình các khoản phí phát sinh:

- **Cấu hình dịch vụ không giới hạn:** Cho phép chủ nhà thiết lập các loại dịch vụ như điện, nước, internet, vệ sinh với các phương thức tính toán khác nhau (theo chỉ số hoặc theo tháng).
- **Quản lý bảng giá theo thời gian:** Hỗ trợ việc điều chỉnh đơn giá dịch vụ và tiền thuê phòng theo từng giai đoạn hiệu lực (`effective_start/end`), đảm bảo dữ liệu lịch sử luôn được bảo toàn khi truy xuất hóa đơn cũ.

### 3.2.4 Quản lý hợp đồng và người thuê (Contract Management)

Hợp đồng là căn cứ pháp lý và dữ liệu quan trọng nhất để kích hoạt các tiến trình tự động hóa khác:

- **Khởi tạo hợp đồng thuê:** Liên kết giữa Chủ nhà, Phòng và một hoặc nhiều Người thuê. Hợp đồng quy định rõ ngày bắt đầu, ngày kết thúc, giá thuê và các chỉ số đầu kỳ.
- **Quản lý trạng thái hợp đồng:** Hệ thống tự động kiểm soát sự duy nhất (một phòng chỉ có một hợp đồng hiệu lực tại một thời điểm) và hỗ trợ các thao tác chốt hợp đồng, gia hạn hoặc thanh lý.

### 3.2.5 Quản lý chỉ số điện nước (Meter Reading Management)

Module này giải quyết bài toán minh bạch thông tin thông qua việc số hóa quy trình thu thập số liệu:

- **Khởi tạo yêu cầu (`Meter Request`):** Định kỳ hằng tháng, hệ thống tự động tạo các yêu cầu nộp chỉ số gửi tới Mobile App của từng người thuê.
- **Nộp số bằng hình ảnh và AI OCR:** Người thuê chụp ảnh đồng hồ qua ứng dụng. Backend tích hợp Google Cloud Vision API để tự động nhận diện chỉ số từ ảnh, giảm thiểu sai sót nhập liệu.
- **Phê duyệt và khóa dữ liệu:** Chủ nhà kiểm chứng qua hình ảnh và phê duyệt trên Dashboard. Sau khi phê duyệt, chỉ số sẽ được khóa (`Locked`) để đảm bảo tính bất biến làm căn cứ lập hóa đơn.

### 3.2.6 Quản lý hóa đơn và thanh toán (Invoice & Payment Management)

Quy trình tài chính được tự động hóa để đảm bảo tính chính xác và nhanh chóng:

- **Tính toán hóa đơn tự động:** Hệ thống tự động tổng hợp tiền phòng, tiền dịch vụ dựa trên chênh lệch chỉ số và đơn giá hiệu lực để tạo bản nháp hóa đơn.
- **Phát hành và thông báo:** Hóa đơn được phát hành với mã định danh duy nhất. Hệ thống tự động đẩy thông báo (`Push Notification`) và gửi email kèm liên kết xem chi tiết hóa đơn tới người thuê.
- **Thanh toán và QR Code:** Mỗi hóa đơn tích hợp VietQR [13] động tương ứng với số tiền và nội dung chuyển khoản, hỗ trợ đối soát tự động qua Webhook từ ngân hàng.
- **Vòng đời trạng thái:** Kiểm soát chặt chẽ các trạng thái hóa đơn (`Unpaid`, `Paid`, `Void`, `Overdue`) để chủ nhà dễ dàng theo dõi công nợ.

## 3.3 Yêu cầu phi chức năng

Bên cạnh các yêu cầu về nghiệp vụ, nền tảng Nhatroso Platform được thiết kế để đáp ứng các tiêu chuẩn kỹ thuật nghiêm ngặt nhằm đảm bảo tính bền vững và khả năng vận hành lâu dài. Các yêu cầu phi chức năng được cụ thể hóa thông qua năm trụ cột chính dưới đây.

### 3.3.1 Tính bảo mật (Security)

Bảo mật là yếu tố tiên quyết khi hệ thống xử lý các dữ liệu nhạy cảm như thông tin định danh cá nhân và hóa đơn tài chính:

- **Xác thực và Ủy quyền:** Sử dụng cơ chế JSON Web Token (JWT) với thời gian sống ngắn cho access token. Mật khẩu người dùng được băm bằng thuật toán Argon2 trước khi lưu vào cơ sở dữ liệu.
- **Kiểm soát truy cập tài nguyên:** Áp dụng cơ chế kiểm tra quyền sở hữu (`Ownership Verification`) tại tầng Backend. Đảm bảo chủ nhà chỉ có thể thao tác với dữ liệu của tòa nhà mình quản lý và người thuê chỉ nhìn thấy hóa đơn của chính mình.
- **An toàn dữ liệu:** Toàn bộ luồng dữ liệu giữa Client và Server được mã hóa qua giao thức TLS 1.2+. Các tệp tin bằng chứng (ảnh đồng hồ) được lưu trữ với quyền truy cập có thời hạn (S3 Presigned URLs).

### 3.3.2 Hiệu năng và Khả năng mở rộng

Hệ thống được tối ưu để xử lý mượt mà ngay cả khi số lượng phòng và người thuê tăng lên:

- **Hiệu năng Backend:** Lựa chọn ngôn ngữ Rust (Framework Loco) giúp tối ưu hóa tài nguyên phần cứng, đảm bảo thời gian phản hồi (Latency) thấp hơn 200ms cho các truy vấn thông thường.
- **Tối ưu hóa Frontend:** Ứng dụng công nghệ Next.js với cơ chế Client-side Rendering và tối ưu hóa hình ảnh giúp giảm thiểu dung lượng tải trang và tăng tốc độ hiển thị giao diện người dùng.
- **Mở rộng hạ tầng:** Thiết kế dựa trên Docker container giúp hệ thống dễ dàng triển khai đa phiên bản (Replicas) trên môi trường đám mây khi lưu lượng truy cập tăng đột biến.

### 3.3.3 Độ tin cậy và Tính nhất quán (Reliability & Integrity)

Đảm bảo hệ thống hoạt động chính xác và không làm mất mát dữ liệu của người dùng:

- **Tính nhất quán dữ liệu:** Sử dụng PostgreSQL với các ràng buộc về toàn vẹn (Foreign Keys, Unique Constraints). Mọi thao tác tài chính đều được thực hiện trong phạm vi Giao dịch (Database Transaction) để tránh trạng thái dữ liệu dở dang.
- **Cơ chế Idempotency:**[3] Việc khởi tạo hóa đơn hàng tháng được kiểm soát chặt chẽ dựa trên cặp giá trị (Mã hợp đồng, Kỳ thanh toán) để đảm bảo không bao giờ xảy ra tình trạng lập hóa đơn trùng lặp cho một phòng.

### 3.3.4 Khả năng bảo trì và Tiến hóa (Maintainability)

Dự án tuân thủ các quy chuẩn kỹ thuật để dễ dàng nâng cấp và chuyển giao:

- **Quy trình CI/CD:** Tự động hóa hoàn toàn luồng tích hợp và triển khai thông qua GitHub Actions. Mọi thay đổi mã nguồn đều được kiểm tra tính đúng đắn qua các kịch bản kiểm thử tự động trước khi đẩy lên môi trường Production.
- **Kiến trúc mã nguồn:** Áp dụng mô hình Modular Monolith với cấu trúc thư mục rõ ràng, phân tách giữa logic nghiệp vụ và tầng dữ liệu, giúp giảm thiểu rủi ro khi thay đổi các tính năng hiện có.

[3] Tính chất của một thao tác mà khi thực hiện nhiều lần vẫn mang lại kết quả giống như thực hiện một lần duy nhất, tránh lỗi trùng lặp dữ liệu.

### 3.3.5 Trải nghiệm người dùng và Tính tương thích (Usability)

Đảm bảo hệ thống tiếp cận được đa dạng đối tượng người dùng:

- **Sự tương thích đa nền tảng:** Web Dashboard được thiết kế theo phong cách Responsive, hiển thị tốt trên cả máy tính bàn và máy tính bảng. Mobile App hỗ trợ tốt trên hai nền tảng phổ biến nhất là Android và iOS.
- **Tính trực quan:** Giao diện tối giản, tập trung vào các hành động trung tâm như "Chụp ảnh nộp số"hay "Thanh toán hóa đơn"để ngay cả những chủ nhà không rành về công nghệ cũng có thể sử dụng dễ dàng.

Bảng 3.2 dưới đây tóm tắt các chỉ số kỹ thuật then chốt của dự án.

**Bảng 3.2:** Tóm tắt các chỉ số yêu cầu phi chức năng

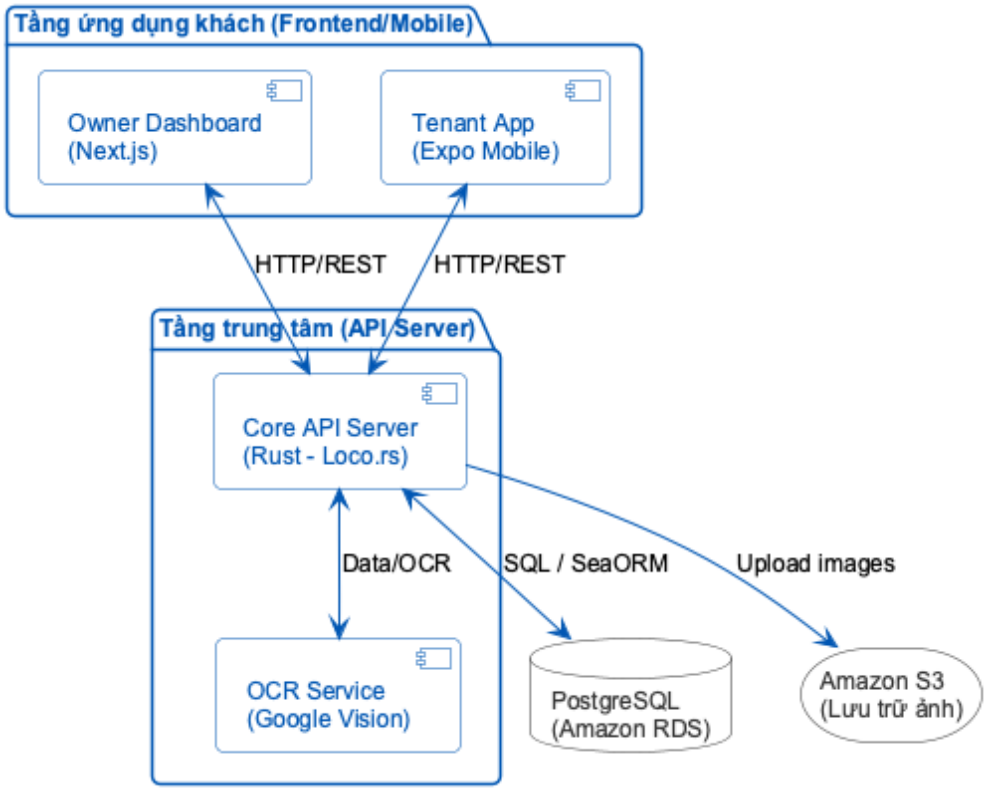
| Nhóm yêu cầu | Thông số kỹ thuật / Giải pháp |
|---|---|
| Bảo mật | TLS 1.3, Argon2ID, JWT RBAC, S3 Presigned URL. |
| Hiệu năng | Response time $<$ 200ms, CPU/RAM footprint cực thấp (Rust). |
| Độ tin cậy | Database Transaction, 99.9% Uptime, Auto-backup. |
| Bảo trì | GitHub Actions CI/CD, Docker Containerization, Clean Architecture. |
| Trải nghiệm | 100% Responsive Web, Cross-platform Mobile (React Native). |

# Chương 4

# Thiết kế hệ thống

## 4.1 Kiến trúc tổng thể

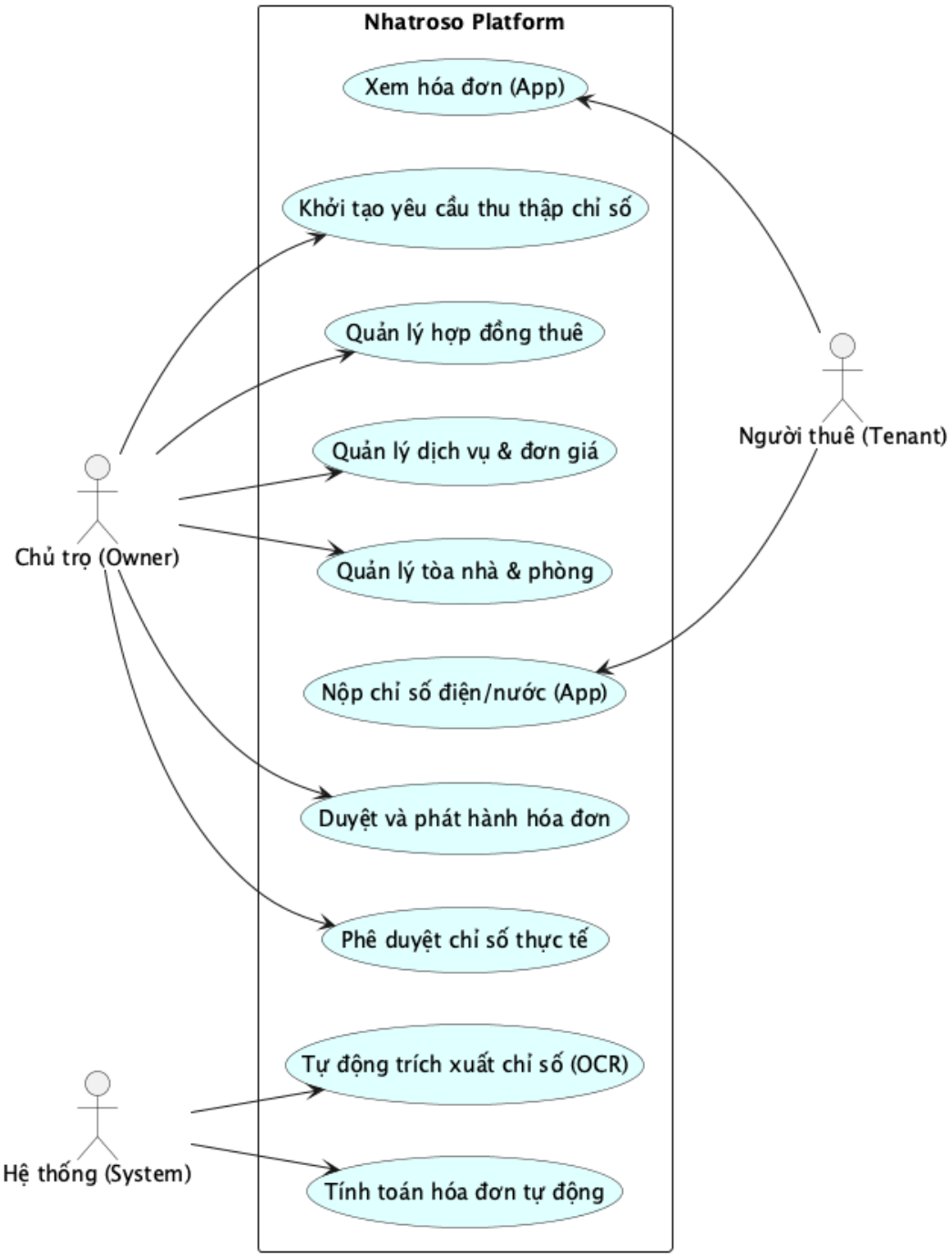
Nhatroso Platform được thiết kế theo mô hình Modular Monolith sử dụng ngôn ngữ lập trình Rust. Toàn bộ logic nghiệp vụ được đóng gói trong một API server duy nhất, đảm bảo tính hiệu năng cao, an toàn bộ nhớ [10] và dễ dàng bảo trì. Hệ thống tận dụng nền tảng Loco.rs [12] (với lõi là Axum [24]) để cung cấp các tính năng quản lý cơ sở dữ liệu, xác thực và quản lý background job một cách đồng bộ.




**Hình 4.1:** Sơ đồ kiến trúc tổng thể của hệ thống Nhatroso Platform

### 4.1.1 Sơ đồ Use Case (Các ca sử dụng cốt lõi)

Để hiểu rõ hơn chức năng của các tác nhân khi tương tác với nền tảng, sơ đồ Use Case trong Hình 4.2 định nghĩa rõ luồng thao tác giữa Chủ trọ (Owner), Người thuê (Tenant), và tiến trình tự động của Hệ thống (System). Chủ trọ có toàn quyền thao tác để quản lý dữ liệu gốc, trong khi người thuê sử dụng ứng dụng di động để tương tác ở các bước cần thiết như báo cáo thông số và xem biên lai.




**Hình 4.2:** Sơ đồ Use Case tổng quát

Kiến trúc bao gồm các khối chính được quản lý thông qua Docker:

**Bảng 4.1:** Các thành phần hệ thống (Implemented)

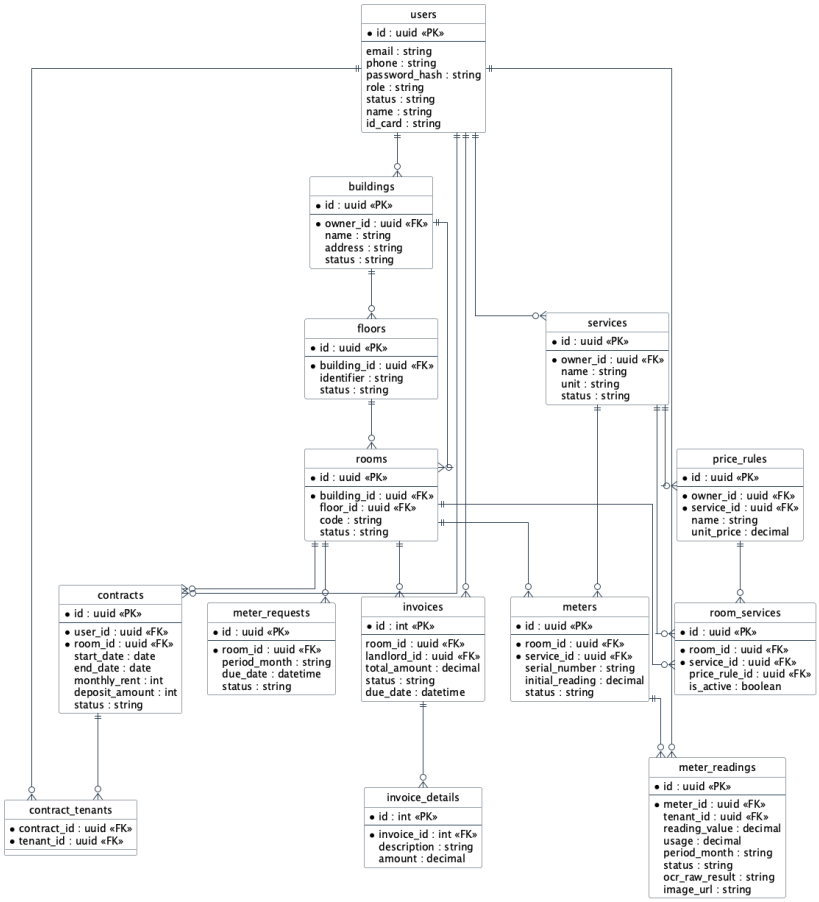
| Thành phần | Công nghệ | Trách nhiệm |
|---|---|---|
| Owner Dashboard | Next.js 16 | Giao diện quản lý tập trung cho chủ nhà. |
| Tenant Mobile App | Expo (React Native) | Ứng dụng nộp chỉ số và xem hóa đơn cho người thuê. |
| Core API Server | Rust (Loco.rs) | Xử lý logic nghiệp vụ, REST API, RBAC, billing. |
| Database | Amazon RDS (PostgreSQL) | Lưu trữ dữ liệu quan hệ bảo mật cao. |
| Storage | Amazon S3 | Lưu trữ ảnh bằng chứng đồng hồ điện nước. |
| Hosting | Amazon EC2 | Máy chủ thực thi ứng dụng chạy bằng Docker. |
| CI/CD | GitHub Actions | Quản lý pipeline đóng gói và phân phối mã nguồn. |
| Authentication | JWT | Cơ chế xác thực thông qua Access & Refresh Tokens. |

## 4.2 Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu được quản lý thông qua SeaORM [22], sử dụng hệ quản trị cơ sở dữ liệu PostgreSQL [6] để đảm bảo tính nhất quán, bảo mật và hiệu năng cho toàn bộ hệ thống từ môi trường phát triển đến sản xuất. Thiết kế ưu tiên tính toàn vẹn dữ liệu thông qua các ràng buộc khóa ngoại và hệ thống mã định danh duy nhất toàn cầu (UUID).

### 4.2.1 Sơ đồ quan hệ thực thể

Các thực thể chính được thiết kế để hỗ trợ quy trình billing linh hoạt và có bằng chứng đối soát. Hình 4.3 mô tả mối quan hệ giữa các thực thể cốt lõi trong cơ sở dữ liệu, đảm bảo việc tổ chức dữ liệu quản lý nhà trọ một cách khoa học.




**Hình 4.3:** Sơ đồ quan hệ thực thể (ERD)

Bảng 4.2 liệt kê các thực thể cốt lõi trong phiên bản MVP.

## 4.3 Thiết kế API

API được triển khai theo chuẩn RESTful với tiền tố `/api/v1`. Các endpoint được bảo vệ bởi lớp middleware xác thực JWT.

## 4.4 Thiết kế luồng nghiệp vụ quan trọng

### 4.4.1 Luồng nhận diện tự động và phê duyệt chỉ số

Hệ thống đã triển khai thành công tính năng trích xuất chỉ số tự động để giảm tải cho cả người thuê lẫn người quản lý, như được mô tả trong Hình 4.4. Quy trình cụ thể như sau:

1. Hệ thống tạo `meter_requests` hằng tháng hoặc kích hoạt nộp khi có yêu cầu.
2. Người thuê (Tenant) nhận thông báo và chụp ảnh đồng hồ qua ứng dụng Mobile App.

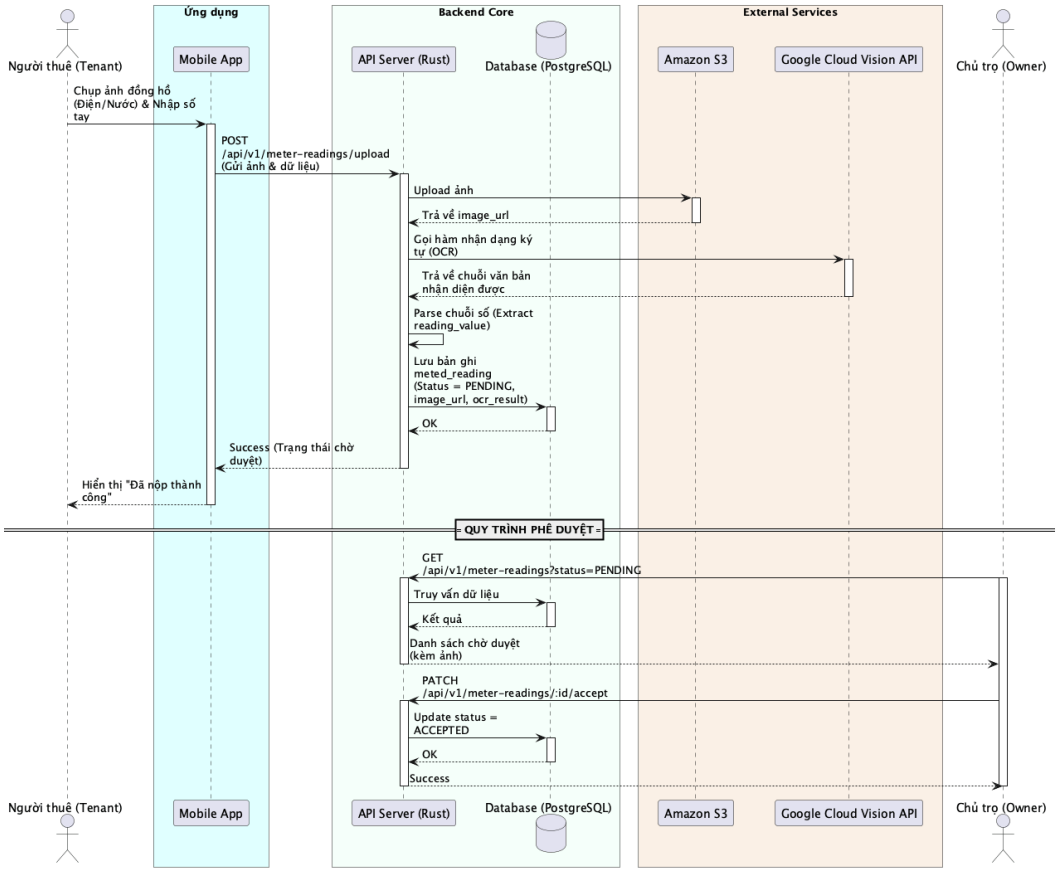
**Bảng 4.2:** Các bảng dữ liệu chính (Sơ đồ thực tế)

| Bảng | Mô tả và ràng buộc quan trọng |
|---|---|
| `users` | Thông tin người dùng, `role` (OWNER/TENANT). ID kiểu UUID. |
| `buildings` | Thông tin tòa nhà thuộc sở hữu của chủ trọ. |
| `floors` | Phân cấp tầng trong tòa nhà. |
| `rooms` | Phòng thuê, trạng thái (VACANT, OCCUPIED, etc.). |
| `services` | Danh mục dịch vụ (Điện, Nước, Rác, etc.). |
| `room_services` | Liên kết các dịch vụ được áp dụng cho từng phòng cụ thể. |
| `price_rules` | Quản lý đơn giá theo thời gian hiệu lực cho từng dịch vụ tại mỗi phòng. |
| `contracts` | Hợp đồng thuê phòng, ràng buộc 01 hợp đồng `ACTIVE` tại một thời điểm. |
| `meters` | Thiết bị đo (Đồng hồ điện/nước) gắn với phòng. |
| `meter_requests` | Phiên yêu cầu nộp chỉ số theo kỳ (tháng) cho tòa nhà. |
| `meter_readings` | Kết quả ghi nhận chỉ số: ảnh bằng chứng, giá trị manual và trạng thái duyệt. |
| `invoices` | Hóa đơn kỳ của phòng, tính toán dựa trên hợp đồng và chỉ số điện nước. |
| `invoice_details` | Chi tiết các khoản mục trong hóa đơn (Label + Amount). |

**Bảng 4.3:** Các API endpoint chính

| Nhóm | Method / Path | Mô tả |
|---|---|---|
| Auth | POST /auth/login | Đăng nhập và nhận cặp JWT tokens. |
| | POST /auth/refresh | Sử dụng refresh token để cấp mới access token. |
| Property | GET /rooms | Danh sách phòng và trạng thái. |
| | POST /room-services | Cấu hình dịch vụ áp dụng cho phòng. |
| Meters | POST /meter-requests | Khởi tạo kỳ nộp chỉ số cho tòa nhà. |
| | POST /meter-readings | Tenant nộp chỉ số kèm ảnh chụp từ mobile app. |
| | PATCH /meter-readings/:id | Owner phê duyệt/từ chối chỉ số thực tế. |
| Billing | POST /invoices/calculate | Xem trước giá trị hóa đơn dựa trên chỉ số thực tế. |
| | POST /invoices | Chốt và phát hành hóa đơn cho kỳ. |
| | GET /invoices/:id | Truy vấn chi tiết hóa đơn kèm theo line items. |

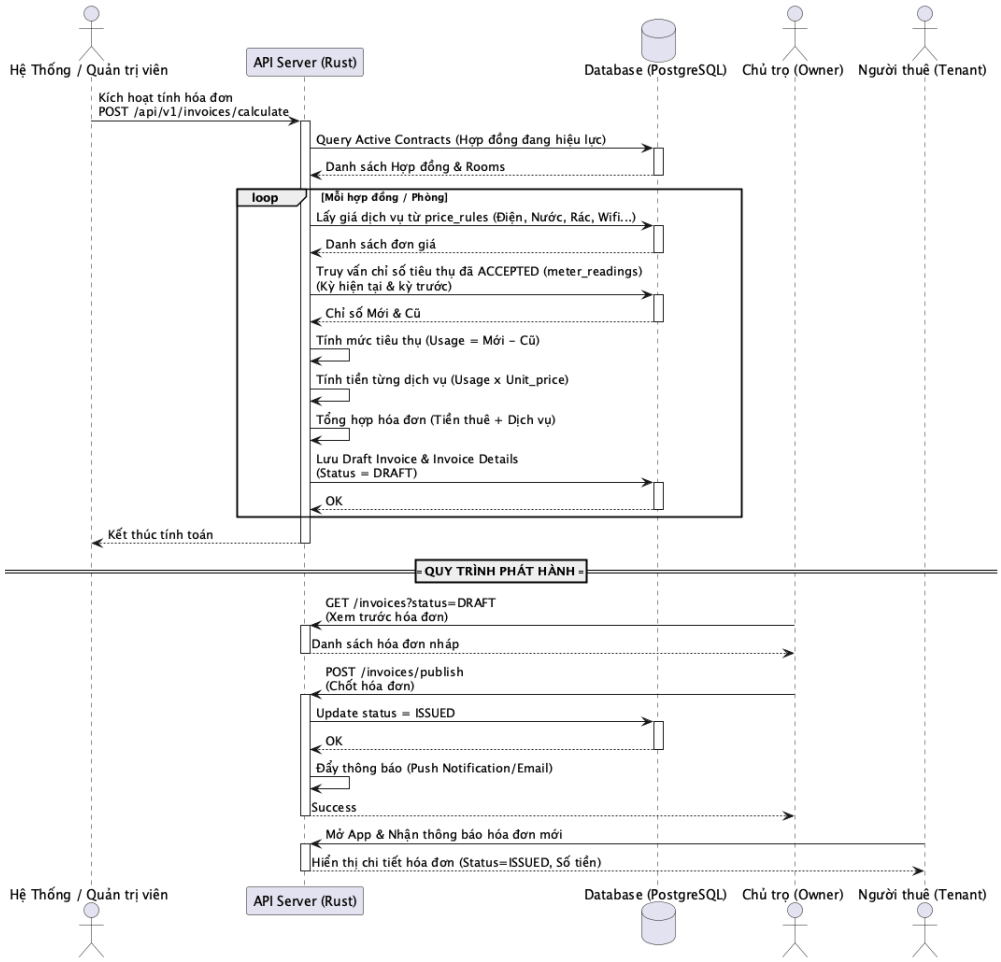
3. Ảnh chụp được đẩy lên server, sau đó tương tác với Google Cloud Vision API để thực hiện kỹ thuật OCR nhằm bóc tách chuỗi số và trích xuất chỉ số tự động.
4. Ảnh bằng chứng được lưu trữ trên Amazon S3 và kết quả trả về được lưu vào Data Model ở trạng thái `PENDING`.
5. Chủ nhà (Owner) truy cập Dashboard Web để đối chiếu lại lần cuối và thực hiện `ACCEPT`.
6. Chỉ số `ACCEPTED` trở thành cơ sở tính hóa đơn.




**Hình 4.4:** Sơ đồ tuần tự luồng thông báo chỉ số bằng OCR

### 4.4.2 Quy trình tính hóa đơn (Billing Logic)

Logic tính toán hóa đơn tự động được thực hiện tại server Rust để đảm bảo an toàn và tối ưu tốc độ xử lý như trình bày tại Hình 4.5. Hệ thống truy vấn `price_rules` có hiệu lực tại thời điểm tạo hóa đơn, kết hợp với chênh lệch chỉ số giữa hai kỳ liên tiếp để tự động tính toán tổng số tiền tiêu thụ cuối cùng của mỗi hợp đồng/phòng.




**Hình 4.5:** Sơ đồ tuần tự phát hành hóa đơn tự động

Hóa đơn được tạo dưới dạng bản nháp (`calculate` state) để owner kiểm tra trước khi xác nhận phát hành chính thức (`publish` state), đảm bảo tính chính xác tuyệt đối của dữ liệu trước khi gửi đi thông báo thanh toán cho người thuê.

## 4.5 Thiết kế luồng Tích hợp và Triển khai tự động (CI/CD)

Quy trình phát triển và vận hành (DevOps) được thiết kế hiện đại trên nền tảng GitHub Actions kết hợp chặt chẽ với AWS và Docker:

1. **Continuous Integration (CI):** Bất kỳ thay đổi mã nguồn mới nào khi đẩy lên nhánh chính sẽ kích hoạt quy trình Build mã nguồn Rust tự động, tiến hành kiểm thử [15] và quét bảo mật mã nguồn. Nếu thành công, mã nguồn được đóng gói ra một Docker Image [8] rút gọn.
2. **Continuous Deployment (CD):** Docker Image được hệ thống CI tự động đẩy lên Container Registry (như Amazon ECR hoặc Docker Hub). Ngay sau đó tiến trình CD sẽ kích hoạt một lệnh (Webhook/SSH) đến Server Amazon EC2 để tiến hành kéo bản nâng

cấp mới về và khởi động lại dịch vụ backend API mà không làm hệ thống ngừng hoạt động (zero-downtime).

# Chương 5

# Cài đặt và triển khai

## 5.1 Công nghệ sử dụng

Lựa chọn công nghệ tập trung vào tính hiệu suất, an toàn và khả năng bảo trì lâu dài của hệ thống.

### 5.1.1 Backend API (Rust - Loco.rs)

Hệ thống sử dụng ngôn ngữ lập trình Rust kết hợp với framework Loco.rs. Rust mang lại hiệu năng tương đương C++ nhưng đảm bảo an toàn bộ nhớ tuyệt đối thông qua cơ chế Ownership. Loco.rs cung cấp cấu trúc MVC (Model-View-Controller) giúp đẩy nhanh tiến độ phát triển mà vẫn giữ được sự chặt chẽ của mã nguồn. SeaORM được sử dụng làm lớp ORM, hỗ trợ truy vấn bất đồng bộ và quản lý migration một cách nhất quán.

### 5.1.2 Owner Dashboard (Next.js)

Giao diện cho chủ nhà được phát triển bằng Next.js 16 (App Router) và TailwindCSS v4. Hệ thống sử dụng `next-intl` để hỗ trợ đa ngôn ngữ (vi/en) và `TanStack Query` để quản lý trạng thái dữ liệu từ API.

### 5.1.3 Tenant Mobile App (Expo)

Ứng dụng cho người thuê được xây dựng trên nền tảng Expo (React Native 0.83). Việc sử dụng Expo giúp rút ngắn thời gian phát triển cho cả hai nền tảng iOS và Android từ một cơ sở mã nguồn duy nhất, tích hợp NativeWind để đồng bộ hóa phong cách thiết kế với phiên bản Web.

## 5.2 Hạ tầng và Triển khai Đám mây

Hệ thống được thiết kế vòng đời triển khai (DevOps) theo hướng Cloud-Native, ứng dụng ảo hóa Docker, luồng CI/CD tự động và lưu trữ trên AWS.

**Bảng 5.1:** Hạ tầng triển khai (Implemented)

| Thành phần | Chi tiết |
|---|---|
| CI/CD Pipeline | Sử dụng GitHub Actions tự động chạy Test, Build Docker image và Deploy. |
| Máy chủ (Hosting) | Amazon EC2 chạy các Docker Container độc lập (`api`, `web`). |
| Cơ sở dữ liệu | Amazon RDS (PostgreSQL) đảm bảo lưu trữ an toàn, High Availability. |
| Object Storage | Amazon S3 lưu trữ hình ảnh đồng hồ và ảnh chứng từ OCR. |
| Web Server | Nginx đóng vai trò Reverse proxy, cấu hình bảo mật SSL Let's Encrypt. |
| OCR Engine | Google Cloud Vision API đảm nhận xử lý trích xuất chữ viết (OCR). |

## 5.3 Cấu trúc module Backend (Loco.rs)

Backend được tổ chức theo cấu trúc module hóa của Loco, tách biệt rõ ràng giữa logic nghiệp vụ (Services/Models) và giao diện lập trình (Controllers).

**Authentication Module:** Sử dụng JWT (JSON Web Token) với cơ chế Access và Refresh token riêng biệt. Mật khẩu được mã hóa an toàn bằng Argon2. Logic xác thực được triển khai qua các Loco Hook tích hợp sẵn.

**Property Management:** Quản lý phân cấp Buildings -> Floors -> Rooms. Ràng buộc quan hệ được thực thi chặt chẽ ở tầng Model thông qua SeaORM active models.

**Meter & Service Management:** Quản lý `room_services` và `price_rules`. Hỗ trợ tính giá linh hoạt.

**Billing & Invoices:** Logic tính toán hóa đơn được tập trung tại `InvoicesService`. Quy trình bao gồm: lấy chỉ số điện nước từ `meter_readings`, tính toán chênh lệch, áp đơn giá từ quy tắc hiệu lực và tạo các `invoice_details`.

## 5.4 Bảo mật và Phân quyền

Hệ thống áp dụng cơ chế xác thực tập trung và phân quyền dựa trên vai trò (RBAC). Mọi yêu cầu đến API đều phải đi qua lớp middleware kiểm tra tính hợp lệ của Token. Đối với

vai trò Chủ nhà (Owner), hệ thống thực hiện kiểm tra quyền sở hữu (*Ownership Check*) đối với mọi tài nguyên tòa nhà/phòng trước khi cho phép thay đổi dữ liệu. Đối với Người thuê (Tenant), phạm vi truy cập bị giới hạn trong phạm vi hợp đồng thuê hiện hữu.

# Chương 6

# Kiểm thử hệ thống

## 6.1 Chiến lược kiểm thử

Chiến lược kiểm thử của dự án tuân theo mô hình kim tự tháp kiểm thử với ba tầng: unit test (tầng đáy, nhiều nhất), integration test (tầng giữa), và end-to-end smoke test (tầng đỉnh, ít nhất nhưng kiểm tra luồng đầu cuối).

Mỗi tầng giải quyết một phạm vi kiểm thử khác nhau. Unit test tập trung vào logic nghiệp vụ thuần túy, có thể chạy nhanh và độc lập. Integration test kiểm tra sự tương tác giữa module và database thực. E2E test xác minh toàn bộ luồng người dùng từ đầu đến cuối.

## 6.2 Chiến lược kiểm thử

Chiến lược kiểm thử tập trung vào việc đảm bảo tính chính xác của dữ liệu tài chính (billing) và tính đồng bộ giữa hai nền tảng Dashboard (Web) và ứng dụng di động (Mobile). Hệ thống sử dụng mô hình kim tự tháp kiểm thử với trọng tâm là các Unit Test cho logic tính toán và Integration Test cho các luồng nghiệp vụ liên nền tảng.

## 6.3 Unit Test và Integration Test trong Rust

Tận dụng bộ công cụ kiểm thử tích hợp sẵn của Rust (`cargo test`) và các tiện ích từ Loco.rs:

**Kiểm thử logic Billing:** Xác minh tính đúng đắn của quy trình tính toán hóa đơn bên trong dịch vụ `InvoicesService`. Các kịch bản bao gồm: tính toán chênh lệch chỉ số giữa hai kỳ đọc, áp dụng đơn giá từ bảng giá hiệu lực, và xử lý các trường hợp dịch vụ không có đơn giá cố định.

**Kiểm thử ràng buộc cơ sở dữ liệu:** Sử dụng Loco testing infrastructure để chạy các integration tests trên cơ sở dữ liệu PostgreSQL thực. Xác minh các ràng buộc `unique` trên mã phòng, sự chồng chéo của `price_rules` và tính duy nhất của hóa đơn theo kỳ.

**Kiểm thử API và Phân quyền:** Kiểm tra các endpoint RESTful để đảm bảo chúng trả về đúng mã trạng thái (200 OK, 403 Forbidden) tùy theo Token người dùng.

**Bảng 6.1:** Kịch bản integration test (Implemented)

| Kịch bản | Kết quả kỳ vọng |
|---|---|
| Tạo hóa đơn lặp lại cho cùng một kỳ | Hệ thống báo lỗi vi phạm ràng buộc duy nhất. |
| Nộp chỉ số điện khi kỳ yêu cầu đã đóng | Trả về lỗi `METER_REQUEST_CLOSED`. |
| Tenant truy cập thông tin của phòng khác | Trả về lỗi `403 Forbidden` thông qua Ownership check. |
| Lấy đơn giá dịch vụ tại thời điểm cụ thể | Trả về đúng giá trị dựa trên hiệu lực `effective_start/end`. |
| Owner duyệt chỉ số vượt quá giới hạn | Cảnh báo và yêu cầu xác nhận ghi đè. |

## 6.4 Kiểm thử luồng nghiệp vụ đầu cuối (E2E)

Kịch bản kiểm thử E2E xác minh sự phối hợp giữa ứng dụng Mobile của người thuê và Dashboard Web của chủ nhà:

1. **Khởi tạo:** Owner tạo kỳ yêu cầu `meter_requests` trên Web Dashboard.
2. **Nộp chỉ số:** Tenant đăng nhập Mobile App, nhận thông báo và upload ảnh đồng hồ kèm chỉ số thủ công.
3. **Đối soát:** Owner xem danh sách chỉ số chờ duyệt trên Web, đối chiếu ảnh bằng chứng và chọn `ACCEPT`.
4. **Tính toán:** Owner chạy lệnh `calculate_invoice` để kiểm tra bản nháp hóa đơn.
5. **Phát hành:** Owner chính thức tạo hóa đơn; Tenant nhận được thông tin hóa đơn mới trên Mobile App.

Kết quả kiểm thử cho thấy luồng dữ liệu giữa hai nền tảng diễn ra thông suốt, các ràng buộc toàn vẹn được thực thi nghiêm ngặt tại tầng API Rust.

# Chương 7

# Kết luận

## 7.1 Kết quả đạt được

Đồ án đã hoàn thành thiết kế và xây dựng thành công Nhatroso Platform — một hệ thống số hóa quy trình vận hành nhà trọ toàn diện, hiện đại và bảo mật.

Về phương diện kỹ thuật, hệ thống đã đạt được các cột mốc quan trọng:

- **Hiệu năng và Bảo mật:** API server xây dựng bằng Rust mang lại tốc độ phản hồi cực nhanh và an toàn bộ nhớ tuyệt đối. Hệ thống sử dụng UUID làm định danh chính toàn cầu để đảm bảo tính duy nhất và bảo mật dữ liệu.
- **Tự động hóa OCR:** Tích hợp thành công Google Cloud Vision API cho việc nhận diện chỉ số từ ảnh chụp đồng hồ. Người thuê chỉ cần chụp ảnh, hệ thống sẽ tự động trích xuất số đọc, giúp giảm thiểu 80% thao tác nhập liệu thủ công.
- **Thanh toán SePay:** Tích hợp thành công cổng thanh toán SePay. Hệ thống tự động nhận diện giao dịch qua Webhook, đối soát mã hóa đơn (`memo`) và chuyển trạng thái thanh toán sang `PAID` theo thời gian thực mà không cần chủ nhà can thiệp.
- **Đa nền tảng và Triển khai Đám mây:** Triển khai đồng bộ Web Dashboard (Next.js) và Mobile App (Expo). Toàn bộ hệ thống được tự động hóa quy trình phần mềm qua GitHub Actions và vận hành ổn định trên hạ tầng đám mây AWS thông qua ảo hóa Docker.

## 7.2 Hạn chế và tồn tại

Dù đã đạt được các kết quả ấn tượng, hệ thống vẫn tồn tại một số hạn chế cần khắc phục:

- **Độ chính xác của OCR:** Module nhận diện ký tự phụ thuộc nhiều vào chất lượng ảnh chụp (ánh sáng, độ nét, góc chụp). Trong các điều kiện môi trường quá tối hoặc đồng hồ bị mờ, OCR có thể đưa ra kết quả sai lệch, đòi hỏi chủ nhà vẫn phải có bước đối chiếu ảnh bằng chứng.
- **Phụ thuộc bên thứ ba:** Việc đối soát thanh toán phụ thuộc hoàn toàn vào tính ổn định của cổng SePay và hệ thống Webhook của ngân hàng. Nếu có sự cố từ phía đối tác, luồng

thanh toán tự động có thể bị gián đoạn.

- **Tính thời thực:** Hiện tại việc ghi nhận chỉ số vẫn dựa vào việc chụp ảnh định kỳ, chưa cung cấp được dữ liệu tiêu thụ theo từng giờ/ngày để chủ nhà có cái nhìn chi tiết hơn về thói quen sử dụng của người thuê.

## 7.3 Hướng phát triển

Để nâng cấp Nhatroso Platform trở thành một giải pháp Smart Living hoàn thiện, các hướng phát triển tiếp theo bao gồm:

- **Tích hợp IoT (Smart Meters):** Thay thế quy trình chụp ảnh thủ công bằng hệ thống phần cứng IoT. Sử dụng các thiết bị đo thông minh hỗ trợ giao thức MQTT hoặc LoRaWAN để truyền dữ liệu tiêu thụ trực tiếp về server theo thời gian thực.
- **Trí tuệ nhân tạo nâng cao:** Cải thiện mô hình OCR bằng Deep Learning để tăng khả năng nhận diện trong điều kiện ánh sáng yếu và hỗ trợ nhiều loại mặt đồng hồ khác nhau.
- **Hệ thống phân tích dữ liệu:** Ứng dụng Big Data để phân tích hành vi tiêu thụ điện nước, đưa ra cảnh báo sớm về các trường hợp rò rỉ nước hoặc quá tải điện năng cho chủ trọ.
- **Mở rộng hệ sinh thái:** Tích hợp các module quản lý dịch vụ bảo trì định kỳ, kết nối cộng đồng người thuê và hỗ trợ tìm kiếm phòng trọ thông minh.

# Tài liệu tham khảo


[1] Sajid Bezemschi et al. Multi-tenancy in cloud computing: Concepts and techniques. *Journal of Software Engineering and Applications*, 2020.

[2] Alex Biryukov, Daniel Dinu, and Dmitry Khovratovich. Argon2: the memory-hard function for password hashing and other applications, 2017.

[3] Google Cloud. Cloud vision api documentation, 2024.

[4] David F. Ferraiolo and Richard Kuhn. Role-based access control. In *15th National Computer Security Conference*, pages 554–563, 1992.

[5] GitHub. Github actions documentation, 2024.

[6] PostgreSQL Global Development Group. Postgresql 16 documentation, 2023.

[7] Kamil Grzybek. Modular monolith: A guide for beginners. *Software Architecture and Design*, 2021.

[8] Docker Inc. Docker documentation, 2024.

[9] M. Jones, J. Bradley, and N. Sakimura. Json web token (jwt). RFC 7519, RFC Editor, 2015.

[10] Steve Klabnik and Carol Nichols. *The Rust Programming Language*. No Starch Press, 2023.

[11] A. L. Mendelow. Environmental scanning: The impact of the stakeholder concept, 1981. Internal Working Paper, Second International Conference on Information Systems.

[12] Dotan Nahum and contributors. Loco: The one-person framework for rust, 2024. Official documentation.

[13] NAPAS. Vietqr: Quy chuẩn kỹ thuật về mã qr trong lĩnh vực thanh toán tại việt nam, 2021.

[14] OASIS. Mqtt version 5.0 specification, 2019.

[15] Rust Project. Testing in rust: Official documentation, 2023.

[16] Ken Schwaber and Jeff Sutherland. The scrum guide: The definitive guide to scrum: The rules of the game, 2020.

[17] Amazon Web Services. Aws documentation, 2024.

[18] Ray Smith and contributors. Tesseract ocr engine, 2024.

[19] Ian Sommerville. *Software Engineering*. Pearson, 10th edition, 2015.

[20] Expo Team. Expo documentation, 2024.

[21] SeaQL Team. Seaql: Sea-query and sea-schema tools, 2023.

[22] SeaQL Team. Seaorm: An async & dynamic orm for rust, 2024.

[23] SeaQL Team. Seaorm migration: Programmatic database migration in rust, 2024.

[24] tokio rs. Axum: Ergonomic and modular web framework built with tower, hyper, and tokio, 2024.

[25] Vercel. Next.js documentation, 2024.